



Universität Stuttgart
Institut für Fördertechnik
und Logistik



Universität Stuttgart
Fakultät Konstruktions-, Produktions- und Fahrzeugtechnik
Institut für Fördertechnik und Logistik

Verspätungsorientiertes Deep Reinforcement Learning für dynamische Auslagerung und Umlagerung in mehrfachtiefen Lagersystemen

Tardiness-Oriented Deep Reinforcement Learning for Dynamic Retrieval and Relocation in Multi-Deep Storage Systems

Research
Thesis
25 July 2026

Author: Zhi Li
Mat.-Nr.: XX XX XX XX

Examiner:
Supervisor: Funing Li

Preface

This manuscript was written during my time at the Institut für Fördertechnik und Logistik. I would like to thank everyone who contributed. In particular, I would like to express our special thanks to M.Sc. Funing Li.

Stuttgart, 25 July 2026

Zhi Li

Contents

1	Introduction	1
2	Related work	4
3	Problem description	5
4	Proposed approach	8
4.1	State representation	8
4.2	Action space	10
4.3	Reward function design	12
4.3.1	Lower-bound derivation for simple retrieval and relocation operations	12
4.3.2	Lower-bound derivation within a single storage lane	15
4.3.3	Layout-level lower-bound derivation	19
4.3.4	State-level total-tardiness lower bound derivation	31
4.3.5	Reward construction based on the state-level lower bound	33
4.3.6	Counterfactual reward shaping	37
4.4	Der Vortrag	40
	Bibliography	42

Chapter 1

Introduction

In recent years, e-commerce has maintained rapid growth worldwide. According to the global Business-to-Consumer (B2C) e-commerce forecast report published by ResearchAndMarkets, the global B2C e-commerce market achieved a compound annual growth rate (CAGR) of 9.5% during 2020–2024. The report further forecasts continued growth over 2025–2029, with the market forecast to increase from USD 7.25 trillion in 2025 to approximately USD 9.21 trillion by 2029, corresponding to a CAGR of 6.2% [1]. This growth has also brought multifaceted challenges to warehousing systems. One significant challenge is that e-commerce-driven order growth increases the demand for storage capacity. However, in metropolitan areas, the scarcity and high cost of land limit the feasibility of simply expanding warehouses to meet this demand [2, 3]. Consequently, for warehouses with limited expansion potential, increasing storage density is a more viable solution.

Multi-deep storage layouts are one common approach for increasing storage density. Compared with traditional single-deep layouts, multi-deep layouts improve floor-space utilization by reducing the proportion of aisle space. With the increasing adoption of automated warehousing systems in recent years, multi-deep layouts have also been incorporated into automated vehicle storage and retrieval systems (AVS/RSs), offering a compact storage solution with high space utilization. However, multi-deep layouts also introduce more complex retrieval logic. If a target item is located deeper in a storage lane and is blocked by other non-target items, the target item can only be retrieved after all blocking non-target items have been moved to empty storage locations (in this paper, we denote this process as “relocation”). Each relocation process involves additional time, which negatively affects the operational efficiency of the system [4]. Fig. 1.1 illustrates the structural differences between single-deep and multi-deep layouts and highlights the trade-off between space utilization and operational efficiency in multi-deep layouts. Another significant challenge stems from the widespread adoption of same-day and next-day delivery services driven by e-commerce. Amazon reported that, in 2025, more than 13 billion items were delivered to Prime members worldwide on the same day or the next day; among them, more than 8 billion items were delivered to Prime members in the United States on the same day or the next day, representing an increase of more than 30% over the previous year [5]. The widespread adoption of such services has raised consumer expectations for delivery speed and delivery-time reliability, making customers more sensitive to late deliveries [6, 7]. From a scheduling perspective, this makes tardiness-related performance increasingly important and places greater fulfillment pressure on warehousing systems. In multi-deep layouts, additional retrieval time caused by relocation further increases the complexity of tardiness-oriented scheduling processes.

Against this background, several recent studies have investigated scheduling problems in multi-deep warehousing systems. Many of these studies employ heuristic ap-

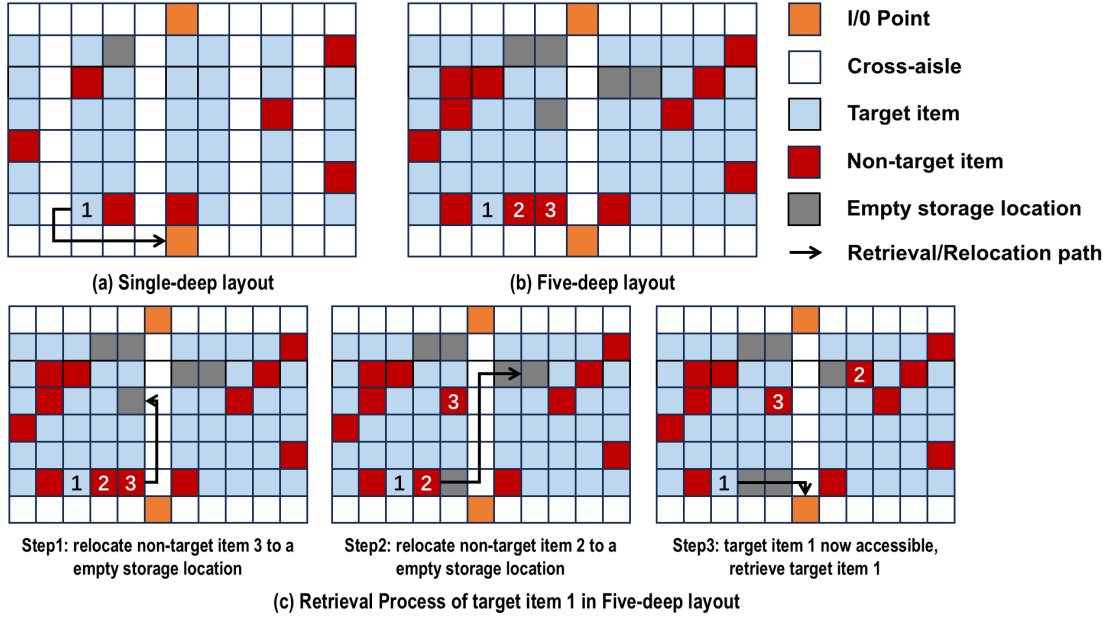


Figure 1.1: Comparison of single-deep and five-deep layouts in terms of space utilization and operational efficiency

proaches [8, 9] or metaheuristic approaches [10, 11]. Although these approaches can produce high-quality solutions for their target problem settings, their performance often relies on problem-specific rules, neighborhood structures, or search operators. When warehouse layouts, task distributions, or operational constraints differ substantially from the setting assumed in the studies, these approaches may require redesign, recalibration, or repeated instance-specific optimization. Therefore, developing scheduling strategies with stronger adaptability and generalizability remains an important research issue in multi-deep warehousing systems.

In response to this research issue, the rapid development of machine learning (ML) in recent years has provided a new perspective. As an important subfield of ML, reinforcement learning (RL) optimizes a decision strategy through interactions with the environment and reward feedback [12]; when combined with deep neural networks (DNNs), deep reinforcement learning (DRL) further improves the ability to learn effective representations from high-dimensional state inputs [13]. Scheduling problems have become an important application domain of DRL, and related studies have covered various complex scenarios, including dynamic task scheduling in stochastic edge-cloud computing environments [14], large-scale task scheduling for multiple unmanned aerial vehicles (UAVs) [15], and dynamic job-shop scheduling under processing-time uncertainty [16]. These studies indicate that DRL can learn effective scheduling strategies in complex dynamic environments and improve real-time decision-making capability. The application of DRL to complex scheduling problems has also driven its adoption in high-density warehousing systems. He et al. adopted a Double & Dueling Deep Q-Network to solve the multi-item retrieval problem in a puzzle-based storage system and demonstrated its superior performance over conventional heuristic strategies [17]. Xu et al. employed a Deep Q-Network to address the real-time transaction scheduling problem in a multi-deep tier-to-tier shuttle-based storage and retrieval system (MDT-SBS/RS), outperforming several classical scheduling strategies under different system configurations [18].

Despite recent progress in scheduling for high-density warehouse systems, most existing studies assume that retrieval requests or target items are known before scheduling starts

[8–11,17]. The study by Li et al. [19] is the most closely related to the problem investigated in this paper, as it also focuses on the retrieval scheduling problem in multi-deep AVS/RSs with the objective of minimizing total tardiness. However, their problem setting assumes a predefined initial target item set: in the initial state, the warehouse is fully occupied by the target items with known individual due dates. In contrast, in real warehouse environments, orders are usually not fully known in advance, but may arrive dynamically and stochastically during the scheduling process. Therefore, it is more realistic for the warehouse to contain non-target items in its initial state. Each arriving order activates one or more initially non-target items and assigns them due dates. Such initially non-target items may be located at different storage depths. They may currently block other target items, but may also be activated by future orders and subsequently become blocked by other items. This dynamic target-item activation mechanism couples retrieval and relocation processes over time and substantially increases the complexity of total tardiness minimization. One exception is Xu et al. [18], who consider stochastic order arrivals in real-time transaction scheduling. However, their optimization objective is the average cycle time rather than total tardiness, and items in the warehouse are not modeled with individual due dates.

Motivated by this gap, this paper investigates the retrieval and relocation scheduling problem in multi-deep AVS/RSs with dynamic order-driven target-item activation, with the objective of minimizing total tardiness. Specifically, this paper extends a graph-based hierarchical DRL framework to this dynamic environment and designs a novel tardiness-oriented reward function to guide the learned strategy toward total tardiness minimization. This problem can be viewed as a dynamic and tardiness-oriented generalization of the classic Block Relocation Problem (BRP), which has been shown to be NP-hard [20]. Unlike the classic BRP, which is typically defined over a static set of blocks, the problem studied here requires coordinated retrieval and relocation decisions under dynamic order-driven target-item activation and individual due-date constraints. Since non-target items may both block currently active target items and later become new target items with due dates, the dynamic evolution of the target item set further increases the combinatorial complexity of the problem. The main contributions of this paper are fourfold:

1. This paper considers a more realistic multi-deep AVS/RS environment with a dynamic order-arrival mechanism. In this environment, initially non-target items can be dynamically and stochastically activated as new target items during operation and assigned due dates, which better reflects real-world warehouse environments where orders arrive over time. This setting requires coordinated retrieval and relocation decisions, which makes the scheduling problem more relevant to real-world order-fulfillment scenarios.
2. This paper designs a tardiness-oriented reward mechanism. A global incremental tardiness penalty is used to capture the increase in tardiness of all currently active target items, and a buffered tardiness mechanism based on static shortest-path distances is designed to reflect potential tardiness risks in the reward signal.
3. This paper extends a graph-based hierarchical DRLdeep reinforcement learning framework of Li et al. [19] to the dynamic multi-deep AVS/RS environment, and integrates Graph Attention Network (GAT)-based encoders to model local topology, blocking relationships, and urgency differences among target items more flexibly.
4. numerical experiments (to be finished)

Chapter 2

Related work

Als Studierender haben Sie prinzipiell die Möglichkeit in einem unserer Studentenarbeitsräume PWR 9 3.146 oder im AR 5b 0.05 zu arbeiten. Falls diese Räume vorübergehend nicht begehbar sind, können Sie dennoch einen Remote-Zugriff auf einen Computer beantragen. Ihr Betreuer organisiert Ihnen den Zugang zu den Rechnern. Bitte beachten Sie nachfolgende Hinweise.

- Sie können sich mit Ihrer st-Nummer (*stXXXXXX@stud.uni-stuttgart.de*) der Universität und dem dazugehörigen Passwort anmelden.
- Mit dem Account wird automatisch ein st-Homes-Verzeichnis angelegt und als Laufwerk verbunden. Dies ist der Arbeitsordner auf den ausschließlich Sie als Benutzer Zugriff haben. Hier sollten sämtliche Daten gespeichert werden. Der Speicherplatz ist auf 10 GB beschränkt.
- Um Dateien mit anderen Studenten und dem Betreuer auszutauschen können Sie einen Austauschordner
`\\inm-cifs.tik.uni-stuttgart.de\shared\safeStud\Nachname` anlegen.
- Umfangreiche Berechnungen sollen lokal unter `D:\scratch` durchgeführt werden. Wichtige Ergebnisse sollten zeitnah nach `Z:\` verschoben werden um Datenverluste zu verhindern.
- Bitte nehmen Sie keine eigenständigen Installationen vor. Sollten Sie zusätzliche Software benötigen, wenden Sie sich bitte an einen der beiden Administratoren¹.

¹admin@inm.uni-stuttgart.de

Chapter 3

Problem description

This work investigates a dynamic retrieval and relocation scheduling problem in a shuttle-based variant of multi-deep AVS/RSs, namely multi-deep Shuttle-Based Storage and Retrieval Systems (SBS/RSs). Specifically, this work focuses on the scheduling problem within a fixed tier served by a single shuttle in a tier-captive multi-deep SBS/RS, considering dynamic order-driven target item activation, with the objective of minimizing total tardiness.

To reflect the compact storage lane configuration constraint with practical multi-deep warehouse operations, items in each storage lane are stored contiguously from the deep end, without empty locations between stored items. Accordingly, in any partially occupied storage lane, all occupied storage locations must be located deeper than all empty storage locations.

In the initial state, the warehouse is not fully occupied. The initial items are distributed randomly across storage locations subject to the above compact storage lane configuration constraint. A subset of the initial items is activated by initially known orders as target items with individual due dates, while the remaining initial items are non-target items.

This random initial configuration may place target items at different storage depths, with some targets located behind one or more non-target items. Before such a target can be retrieved, the blocking non-target items must first be relocated. At each decision point, the shuttle either retrieves an accessible target item by transporting it to the I/O point or relocates an accessible blocking non-target item to a feasible empty location in another storage lane.

During the scheduling process, orders may arrive at any discrete time step, and each order may contain multiple item requests. Each item request converts one currently non-target item into a new target item and assigns it an individual due date. To model the stochastic arrival of such item requests, this work adopts a global Poisson arrival process, which is used to describe the number of random arrival events occurring independently in non-overlapping time intervals.

Fig. 3.1 illustrates the general logic of this dynamic order-driven target item activation. Specifically, let X_t denote the number of non-target items requested to be activated by orders arriving at discrete time step t . Then, X_t follows a Poisson distribution, and its probability mass function (PMF) is given by:

$$\Pr(X_t = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k \in \mathbb{N}_0 \quad (3.1)$$

where the non-negative integer k denotes one possible value of the number of items requested to be activated in this time step, and the parameter λ denotes the expected number of activations per time step. Let $\mathcal{N}_t$ denote the set of all non-target items in the

warehouse at discrete time step t . Then, the actual number of activated items A_t is given by:

$$A_t = \min\{X_t, |\mathcal{N}_t|\}. \quad (3.2)$$

After A_t is determined, the system uniformly samples A_t non-target items from the set $\mathcal{N}_t$ without replacement, activates them as new target items, and randomly assigns individual due dates to them.

To ensure a finite dynamic activation horizon and comparable scheduling episodes across problem instances, a maximum activation time window and a threshold for the number of remaining non-target items are predefined. The target item activation process terminates when either the activation time window expires or the number of remaining non-target items falls below the predefined threshold. Once either condition is met, no further target items are activated.

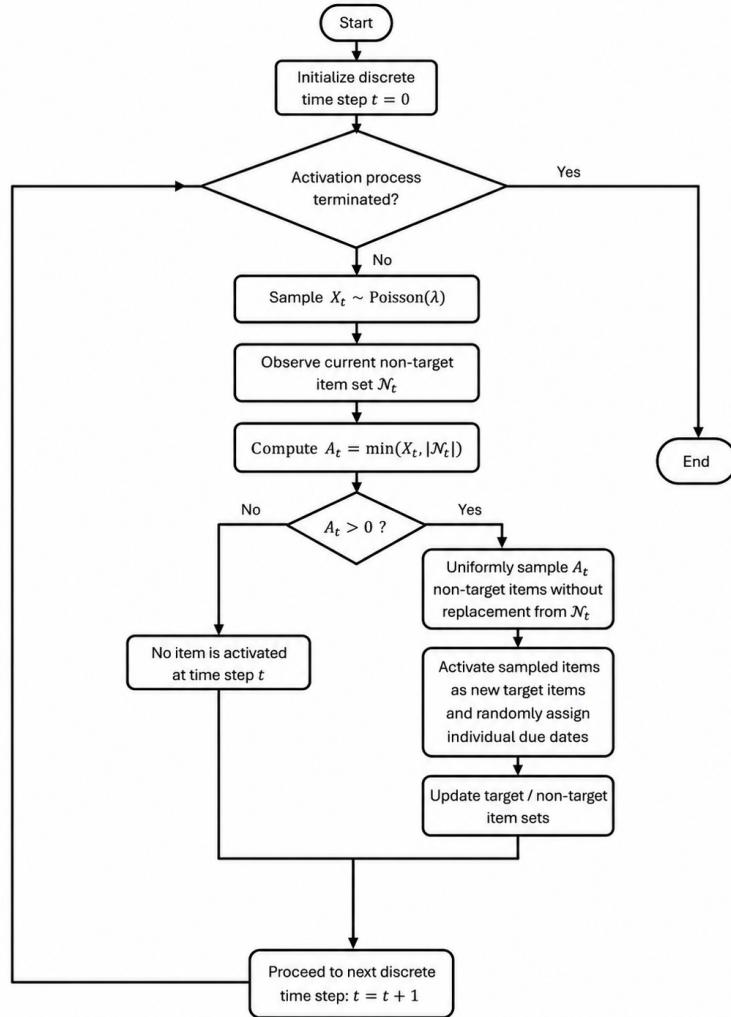


Figure 3.1: Flowchart of dynamic order-driven target item activation

The optimization objective of this work is to minimize the total tardiness of all activated target items. Let $\mathcal{T}$ denote the set of all activated target items during the entire scheduling process. For any target item $i \in \mathcal{T}$, let D_i denote its due date and let C_i denote the discrete time step at which it completes unloading at an I/O point. Then, the

tardiness of this target item is defined as:

$$T_i = \max\{0, C_i - D_i\}. \quad (3.3)$$

Therefore, the mathematical expression for minimizing total tardiness is given by:

$$\min \sum_{i \in \mathcal{T}} T_i = \min \sum_{i \in \mathcal{T}} \max\{0, C_i - D_i\}. \quad (3.4)$$

To clearly focus on the objective of this work, we have introduced the following assumptions:

1. This work adopts a discrete-time model. Each one-step movement of the shuttle between two adjacent locations, each loading operation, and each unloading operation consumes one time step.
2. This work considers only the scheduling problem within a fixed tier. Lifts, conveyors, and other auxiliary resources are assumed to be sufficiently configured. Therefore, cross-tier transportation, waiting for auxiliary resources are not considered.
3. All items have the same size, each storage location can store at most one item, and the shuttle can transport at most one item at each retrieval or relocation operation. After a target item is unloaded at an I/O point, it will be immediately removed from the system and regarded as retrieved.
4. The dynamic target item activation process terminates when either of the following conditions is triggered: the current time step exceeds the predefined activation time window, or the number of remaining non-target items in the warehouse falls below a predefined threshold.

Chapter 4

Proposed approach

To ensure the feasibility of applying Deep Reinforcement Learning (DRL), the dynamic retrieval and relocation scheduling problem is formulated as a Markov Decision Process (MDP), represented by a five-tuple $\{\mathcal{S}, \mathcal{A}, R, \gamma, p\}$ where $\mathcal{S}$ denotes the observation (state) space, $\mathcal{A}$ denotes the action space, R denotes the reward function, γ is the discount factor used to balance immediate and long-term rewards, and p denotes the transition probability that characterizes the dynamics of the environment.

A fundamental assumption underlying an MDP is the Markov property, which states that the future evolution of the system depends only on the current state and is independent of the preceding state-action history, given the current state. On the one hand, this property eliminates the need for the agent to retain the complete historical trajectory from the initial state to the current state. On the other hand, provided that the current state s contains sufficient information, the agent can determine an action a solely based on s , thereby making large-scale computation and policy optimization tractable.

Within the environment formulated as an MDP, the DRL agent continuously interacts with the environment by selecting actions and receives feedback through the reward function to optimize its policy. Specifically, at each decision point t , the agent observes the current state s_t from the environment and uses s_t as the input to its policy π , which selects an action a_t . The execution of a_t causes the environment to transition from s_t to the subsequent state s_{t+1} , while the environment provides the agent with a reward r_t . Through repeated interactions with the environment, the agent progressively improves its policy, with the objective of maximizing the expected cumulative reward over an episode.

In this study, the agent is constructed using a hybrid neural network architecture that integrates a Graph Neural Network (GNN) and a Transformer, and the network parameters are optimized using the Proximal Policy Optimization (PPO) algorithm. The graph-based state representation and the scalable action space are introduced in Sections 4.1 and 4.2, respectively. The specific neural network architecture is described in Section 4.3. Section 4.4 provides a detailed description of the reward function design. The PPO mechanism for updating the network parameters is presented in Section 4.5.

4.1 State representation

At each upper-level decision point, the warehouse is represented as an undirected graph and provided as input to the deep reinforcement learning (DRL) agent. Each node in the graph corresponds to a physical position in the warehouse layout, including a storage location, a cross-aisle position, or an I/O point. Edges between adjacent nodes represent the physical connectivity of the warehouse. Adjacent storage locations within the same storage lane are sequentially connected, the entrance of each lane is connected to the cross-

aisle, and cross-aisle nodes are connected according to the physical warehouse layout. In this way, the graph explicitly captures the spatial topology of the multi-deep warehouse as well as the structural dependencies induced by storage depth within each lane. It should be noted that items are not modeled as separate graph nodes. Instead, each storage location is represented by a fixed node, while whether the location currently contains a target item, a non-target item, or is empty is encoded by its node features.

At each state, every node is described by a nine-dimensional feature vector. The first three dimensions jointly represent the physical node type. The first dimension indicates whether the node is a storage location, the second dimension indicates whether it is a cross-aisle node, and the third dimension indicates whether it is an I/O point. The fourth and fifth dimensions represent the vertical and horizontal coordinates of the node in the warehouse layout, respectively, thereby providing the agent with spatial position information.

The sixth dimension describes the current task attribute of a storage location. If the node contains an activated target item, this dimension represents the due date of the corresponding item. If the node contains a non-target item that has not yet been activated, the value is set to -1 . If the storage location is empty, or if the node is not a storage location, the value is set to 0 . Therefore, this feature jointly provides information about item occupancy, whether the stored item is currently a target item, and the due-date information of active target items.

The seventh dimension represents the current position and carrying state of the shuttle. If the shuttle is located at the node while carrying an item, the value is set to 1 . If the shuttle is located at the node without carrying an item, the value is set to -1 . All other nodes take the value 0 . This feature allows the agent to identify the shuttle position from the current state and to distinguish whether the current upper-level decision is made during the unloaded item-selection stage or during the loaded delivery or relocation stage.

The eighth dimension represents the current environment time. Together with the due dates of the target items, this information characterizes the temporal status of the current scheduling problem and enables the agent to distinguish the urgency of different target items at the current decision point.

The ninth dimension provides additional scheduling information associated with the adopted reward model, and its specific meaning therefore varies across reward formulations. Under the original tardiness reward, this dimension represents the buffered tardiness snapshot of an active target item. This quantity is constructed from the due date of the target item, its static shortest path to the nearest I/O point, and the current time, thereby reflecting its tardiness status while accounting for the potential transportation time. For non-target items, empty storage locations, cross-aisle nodes, and I/O points, this feature is set to 0 .

For the three lower-bound-based reward models, the ninth dimension no longer represents buffered tardiness and is instead replaced by target-specific lower-bound information derived from the current physical warehouse state. Under `simple_shuttle_LB`, the ninth dimension represents the required-work lower bound q of the corresponding target item. This quantity consists of the retrieval-operation lower bound required to complete the target item together with the relocation-operation lower bounds of the blocking non-target items currently associated with that target, and therefore estimates the minimum amount of work required to complete the target item.

Under `mixed_shuttle_LB` and `topology_shuttle_LB`, the ninth dimension represents the lane-prefix cumulative work lower bound R of the corresponding target item. Within each storage lane, R is obtained by cumulatively aggregating the q values from the first target item in the lane to the current target item. Consequently, R further encodes the

lane-level precedence relationships induced by the multi-deep storage topology. For all lower-bound-based reward models, the ninth dimension contains the corresponding lower-bound information only for currently active target-item nodes and is set to 0 for all other nodes.

Accordingly, although all reward models retain the same nine-dimensional node-feature structure, the ninth dimension follows different node-feature schemas according to the adopted reward model. The tardiness reward uses buffered tardiness, `simple_shuttle_LB` uses q , whereas `mixed_shuttle_LB` and `topology_shuttle_LB` use R . This design keeps the dimensionality of the state input unchanged while allowing the scheduling lower-bound information associated with the corresponding reward formulation to be incorporated directly into the agent’s state representation.

In addition to the node-level features, the current state contains an independent graph-level dynamic context that indicates whether the dynamic target activation process is currently enabled. This variable takes the value 1 when the dynamic activation process has not exceeded its predefined termination time and at least two non-target items remain unactivated in the warehouse; otherwise, it takes the value 0. The dynamic context does not occupy an additional dimension in the nine-dimensional node feature vector. Instead, it is provided as independent graph-level state information and is fused with the embedding of the node occupied by the shuttle. This enables the agent to explicitly distinguish between the dynamic phase in which further target activations may still occur and the phase in which the dynamic activation process has terminated.

During the dynamic order process, target activation is represented by updating the state of an existing storage-location node rather than modifying the physical graph topology. Once a non-target item is activated by a newly arriving order, its task attribute changes from the inactive non-target state to an active target state with an associated due date, which is reflected in the sixth node feature. At the same time, its ninth feature is updated according to the currently adopted reward model to represent the corresponding buffered tardiness, q , or R . Hence, the static physical structure of the warehouse is primarily represented by the graph nodes and edges, whereas the dynamically evolving item status, due dates, shuttle state, current time, dynamic activation state, and lower-bound information are represented through the node-level and graph-level features.

4.2 Action space

To enable the deep reinforcement learning (DRL) agent to jointly handle the retrieval of target items and the relocation of blocking non-target items, a state-dependent upper-level action space is constructed according to the current shuttle loading state and the task attribute of the carried item. At each upper-level decision point, an action selected by the DRL agent does not directly correspond to an atomic shuttle movement. Instead, the agent selects an upper-level destination node from the currently feasible candidate nodes, while the subsequent movement is handled by the lower-level path planning and execution module.

For a state s_t , the upper-level action space is defined as

$$\mathcal{A}(s_t) = \begin{cases} \mathcal{A}_{\text{item}}(s_t), & \text{if the shuttle is unloaded,} \\ \mathcal{A}_{\text{I/O}}, & \text{if the shuttle carries a target item,} \\ \mathcal{A}_{\text{empty}}(s_t), & \text{if the shuttle carries a non-target item.} \end{cases}$$

where $\mathcal{A}(s_t)$ denotes the complete upper-level action space in state s_t , $\mathcal{A}_{\text{item}}(s_t)$ denotes the set of item nodes that can be selected when the shuttle is unloaded, $\mathcal{A}_{\text{I/O}}$ denotes the

set of selectable I/O points, and $\mathcal{A}_{\text{empty}}(s_t)$ denotes the set of empty storage locations that can be selected as relocation destinations.

When the shuttle is unloaded, the action space consists of currently directly accessible item nodes that are relevant to the scheduling process. Specifically, $\mathcal{A}_{\text{item}}(s_t)$ contains two types of candidate items. The first type consists of directly accessible target items, whose selection initiates the corresponding retrieval operation. The second type consists of directly accessible non-target items that block active target items located deeper in the same storage lane. Selecting such an item initiates the relocation operation required to release the corresponding target item. Accordingly, the candidate set can be expressed conceptually as

$$\mathcal{A}_{\text{item}}(s_t) = \mathcal{A}_{\text{target}}(s_t) \cup \mathcal{A}_{\text{blocker}}(s_t),$$

where $\mathcal{A}_{\text{target}}(s_t)$ denotes the set of currently directly accessible target-item candidates, and $\mathcal{A}_{\text{blocker}}(s_t)$ denotes the set of blocking non-target-item candidates that must be relocated to access active target items located behind them.

Here, “directly accessible” means that the shuttle can access the corresponding item without first moving any other occupied item. Due to the access constraints within a multi-deep storage lane, a target item blocked by other occupied items is not directly included in the current action space. Instead, the blocking non-target item located in front of it must first be processed. In contrast, if a directly accessible non-target item does not block any active target item, relocating it does not contribute to the current retrieval objective and it is therefore excluded from the current feasible action set.

In addition, a blocking non-target item constitutes a feasible candidate action only when a feasible relocation destination exists. Consequently, $\mathcal{A}_{\text{item}}(s_t)$ contains only upper-level actions that are executable in the current state and contribute to the subsequent retrieval process.

When the shuttle carries a target item, the next upper-level decision is to deliver the item to an I/O point. The action space therefore becomes

$$\mathcal{A}(s_t) = \mathcal{A}_{\text{I/O}},$$

where $\mathcal{A}_{\text{I/O}}$ consists of the selectable I/O points. The DRL agent selects one of these I/O points as the delivery destination of the currently carried target item. Since different I/O points may correspond to different travel distances, I/O-point selection itself constitutes part of the retrieval scheduling decision.

When the shuttle carries a blocking non-target item, the item cannot be delivered to an I/O point and must instead be relocated to a feasible empty storage location in another storage lane. In this case, the action space becomes

$$\mathcal{A}(s_t) = \mathcal{A}_{\text{empty}}(s_t).$$

To satisfy the compact storage lane constraint adopted in this work, the relocation destination is restricted to the deepest accessible empty location in each feasible destination lane. For a partially occupied lane, this location corresponds to the deepest empty position in front of the current occupied block that preserves contiguous storage within the lane. For a completely empty lane, it corresponds to the deepest storage location in that lane. Consequently, each feasible destination lane with remaining capacity contributes at most one relocation candidate.

Furthermore, the relocation must be performed to a storage lane different from the source lane of the blocking non-target item. Therefore, the source lane is excluded from

the feasible relocation destinations for the current blocker. The relocation action space can thus be expressed as

$$\mathcal{A}_{\text{empty}}(s_t) = \{e_l \mid l \neq l_{\text{src}}, e_l \text{ is the deepest accessible empty location in lane } l\},$$

where e_l denotes the deepest accessible empty location in destination lane l , and l_{src} denotes the source storage lane of the currently carried blocking non-target item. This design satisfies the compact storage lane constraint while preserving the relocation scheduling flexibility among different destination lanes, allowing the DRL agent to select an appropriate relocation destination according to the current warehouse state.

Accordingly, a complete upper-level item-processing process follows one of two basic decision sequences depending on the item type:

target item $\rightarrow$ I/O point,

or

blocking non-target item $\rightarrow$ empty storage location.

The former corresponds to a retrieval operation, whereas the latter corresponds to a relocation operation. Both operations consist of two consecutive upper-level decisions: the item to be processed is first selected while the shuttle is unloaded, after which the corresponding destination is selected according to the task attribute of the loaded item. In this way, retrieval and relocation can be handled within a unified upper-level node-selection framework.

Since the warehouse layout, current occupancy, active target set, blocking relationships, and shuttle loading state evolve throughout the scheduling process, the sizes of $\mathcal{A}_{\text{item}}(s_t)$ and $\mathcal{A}_{\text{empty}}(s_t)$ also vary dynamically with the state. Therefore, the proposed action space is inherently a variable-size action space rather than a discrete action space with a fixed number of categories. At each upper-level decision point, the DRL agent only needs to select an action from the currently feasible candidate-node set.

This state-dependent action-space design excludes physically infeasible nodes and nodes that do not contribute to the current scheduling objective, thereby focusing the policy search on meaningful retrieval and relocation decisions. Moreover, since each action corresponds to a feasible node under the current state rather than to a predefined fixed action index, the same action representation can naturally accommodate different numbers of storage lanes, different lane depths, and candidate sets that continuously change with dynamic target activation.

4.3 Reward function design

4.3.1 Lower-bound derivation for simple retrieval and relocation operations

The overall scheduling process can be decomposed into a sequence of individual retrieval and relocation operations. To derive a lower bound on the completion time of these operations, several constraints associated with the actual execution process are relaxed, while only the minimum movements and loading/unloading operations that are unavoidable for completing the corresponding operation are retained. Based on this relaxation, lower bounds on the execution times of individual retrieval and relocation operations are derived as follows.

For a target item i stored in the warehouse, a basic retrieval operation consists of the following processes: the shuttle reaches the storage location of target item i in its storage lane and loads the item, transports the loaded item to an Input/Output (I/O) point, and finally unloads the item at the I/O point. Let d_i denote the depth of the storage location occupied by target item i within its storage lane. Accordingly, d_i is also equal to the minimum travel distance from the entrance of this storage lane to the storage location of target item i . To obtain an optimistic estimate of the retrieval time, the travel required for the shuttle to reach the entrance of the storage lane is relaxed, and d_i is therefore used as a lower bound on the travel distance required for the shuttle to reach the storage location of target item i .

Furthermore, let $d^{I/O}$ denote the static shortest-path distance from the storage location of target item i to the nearest I/O point. This distance is used as a lower bound on the travel distance required to transport target item i from its storage location to an I/O point after loading. Let v^{shuttle} denote the traveling speed of the shuttle, τ^{load} the time required for one loading operation, and τ^{unload} the time required for one unloading operation. The theoretical lower bound on the retrieval operation time can then be expressed as

$$\tau^{\text{retrieval}} = \frac{d_i}{v^{\text{shuttle}}} + \tau^{\text{load}} + \frac{d^{I/O}}{v^{\text{shuttle}}} + \tau^{\text{unload}}. \quad (4.1)$$

In Eq. (4.1), d_i/v^{shuttle} represents the minimum travel time required for the shuttle to move from the entrance of the storage lane to the storage location of target item i , while $d^{I/O}/v^{\text{shuttle}}$ represents the minimum travel time required to transport the loaded target item from its storage location to the nearest I/O point. The terms τ^{load} and τ^{unload} represent the time required for the loading and unloading operations, respectively.

In the environment considered in this study, the shuttle moves by one adjacent location per time unit. Therefore, its traveling speed is defined as

$$v^{\text{shuttle}} = 1.$$

Moreover, each loading and unloading operation requires one time unit, i.e.,

$$\tau^{\text{load}} = \tau^{\text{unload}} = 1.$$

By substituting these environment-specific settings into Eq. (4.1), the retrieval operation time lower bound is simplified to

$$\boxed{\tau^{\text{retrieval}} = d_i + d^{I/O} + 2}. \quad (4.2)$$

The constant term 2 in Eq. (4.2) corresponds to one loading operation and one unloading operation. It should be noted that this lower bound is not intended to predict the actual completion time of the retrieval operation. Instead, it provides an optimistic estimate by neglecting additional travel that may be required before the shuttle reaches the entrance of the corresponding storage lane, waiting time caused by other tasks, and other factors that may increase the actual retrieval time. Consequently, $\tau^{\text{retrieval}}$ represents a lower bound on the time required to complete the retrieval operation rather than an estimate of its actual execution time.

The formulation in Eq. (4.2) corresponds to the general case in which target item i is still stored at its original storage location and has not yet been loaded. The lower-bound calculation is further adapted to intermediate system states. If the shuttle is already located at the storage location of target item i , the approach movement represented by d_i has already been completed and is therefore excluded from the remaining operation-time lower bound. Furthermore, if target item i has already been loaded onto the shuttle,

both the approach movement and loading operation have already been completed. In this case, only the minimum travel to the nearest I/O point and the final unloading operation remain. These state-dependent adjustments provide a tighter estimate of the remaining retrieval time without changing the underlying relaxation principle.

For a relocation operation, consider a blocking non-target item j that prevents the retrieval of a target item located deeper in the same storage lane. Let d_j denote the depth of the storage location occupied by blocking non-target item j within its original storage lane. To complete the relocation operation, the shuttle must at least perform the following processes: it first travels from the entrance of the original storage lane to the storage location of blocking non-target item j and loads the item; it then transports the loaded item out of the original storage lane; subsequently, it enters a feasible destination storage lane; and finally, it unloads the blocking item at a feasible storage location in the destination lane.

Similar to the retrieval operation, the actual relocation process is relaxed to obtain a lower bound that does not overestimate the required operation time. In particular, the actual relocation destination of blocking non-target item j is not explicitly determined when calculating the relocation operation time lower bound. Instead, it is assumed that a feasible destination storage lane with the minimum possible entry cost is available. Moreover, the additional cross-lane travel distance between the entrance of the original storage lane and the entrance of the actual destination storage lane is neglected. Let $d^{\min, \text{entry}}$ denote the minimum unavoidable travel distance required for the shuttle carrying the blocking item to enter a feasible destination storage lane. The theoretical lower bound on the relocation operation time is then expressed as

$$\tau^{\text{relocation}} = \frac{d_j}{v^{\text{shuttle}}} + \tau^{\text{load}} + \frac{d_j}{v^{\text{shuttle}}} + \frac{d^{\min, \text{entry}}}{v^{\text{shuttle}}} + \tau^{\text{unload}}. \quad (4.3)$$

In Eq. (4.3), the first term d_j/v^{shuttle} represents the minimum travel time required for the shuttle to approach blocking non-target item j from the entrance of its original storage lane. The second term d_j/v^{shuttle} represents the minimum travel time required to transport the loaded blocking item completely out of its original storage lane. The term $d^{\min, \text{entry}}/v^{\text{shuttle}}$ represents the minimum unavoidable travel time required to enter a feasible destination storage lane. Finally, τ^{load} and τ^{unload} correspond to the loading and unloading operations, respectively. Since the cross-lane travel distance between the original storage lane and a specific destination storage lane is completely relaxed, this distance is not included in the lower-bound calculation.

Under the environment settings considered in this study, $v^{\text{shuttle}} = 1$, and both loading and unloading require one time unit. In addition, under the relaxed relocation model, entering a feasible destination storage lane requires at least one movement to an adjacent location. Therefore,

$$d^{\min, \text{entry}} = 1.$$

By substituting

$$v^{\text{shuttle}} = 1, \quad \tau^{\text{load}} = 1, \quad \tau^{\text{unload}} = 1, \quad d^{\min, \text{entry}} = 1$$

into Eq. (4.3), the relocation operation time lower bound is simplified to

$$\boxed{\tau^{\text{relocation}} = 2d_j + 3}. \quad (4.4)$$

In Eq. (4.4), the term $2d_j$ accounts for the minimum travel required to first approach blocking non-target item j and subsequently transport it out of its original storage lane.

The constant term 3 consists of one loading operation, one minimum movement required to enter a feasible destination storage lane, and one unloading operation.

Similar to the retrieval operation time lower bound, Eq. (4.4) does not determine the actual relocation destination of blocking non-target item j , nor does it include the additional travel distance that may be required between the original storage lane and the actual destination lane. Hence, $\tau^{\text{relocation}}$ represents an optimistic lower bound on the time required to complete the relocation operation.

The lower-bound calculation is also adapted when blocking non-target item j has already been loaded onto the shuttle. In such an intermediate state, the approach movement toward item j and its loading operation have already been completed and are therefore excluded from the remaining relocation time lower bound. Only the unavoidable remaining movements required to leave the original storage lane, enter a feasible relocation lane, and unload the item are retained.

Through the above relaxations, a minimum operation time can be derived for both the retrieval of each target item and the relocation of each blocking non-target item. These individual operation-time lower bounds subsequently serve as the basic components for constructing the state-level lower bound. In the following derivation, the unavoidable relocation work caused by blocking non-target items and the precedence relationships among target items imposed by the multi-deep storage-lane topology are further incorporated to obtain a lower bound on the total tardiness of the current system state.

4.3.2 Lower-bound derivation within a single storage lane

In the previous subsection, the retrieval operation time lower bound $\tau^{\text{retrieval}}$ for an individual target item and the relocation operation time lower bound $\tau^{\text{relocation}}$ for an individual blocking non-target item were derived by relaxing part of the movement and scheduling constraints involved in the actual execution process. However, in a multi-deep storage lane, the retrieval operation of a target item generally cannot be completed independently.

Although different scheduling policies may generate different global operation sequences over the entire warehouse layout, the necessary access precedence among the items currently stored within a given multi-deep storage lane is determined by its Last-In, First-Out-type (LIFO-type) access constraint and is therefore independent of the upper-level scheduling policy. Specifically, because a storage lane can only be accessed from its lane entrance, an item located deeper in the lane cannot be retrieved or relocated before the items blocking access to it have been removed. Consequently, although different scheduling policies may change the operation sequence across different storage lanes as well as the actual execution time of individual operations, the necessary in-lane access precedence induced by the current depth ordering of the items remains unchanged.

Based on this property, the local precedence structure within each storage lane is further exploited to aggregate the individual retrieval and relocation operation time lower bounds derived in the previous subsection and construct lane-level lower bounds. Specifically, a target-specific required-work lower bound q is first constructed for each target item to represent the minimum amount of work required to retrieve the target item together with the necessary relocations of its preceding blockers. A precedence-constrained cumulative work lower bound R is subsequently constructed to represent the minimum cumulative work required to complete a given number of target items while respecting the necessary in-lane access precedence.

Consider an arbitrary storage lane ℓ . All occupied storage locations in this lane are scanned from the lane entrance toward the deeper end of the lane. The target items encountered during this scan are indexed according to their scanning order as

$$i_{\ell,1}^{\text{target}}, i_{\ell,2}^{\text{target}}, \dots, i_{\ell,n_\ell}^{\text{target}},$$

where ℓ denotes the index of the storage lane, n_ℓ denotes the number of target items currently contained in storage lane ℓ , and $i_{\ell,r}^{\text{target}}$ denotes the r -th target item encountered when scanning from the lane entrance toward the deeper end of the lane. Accordingly, a smaller value of r indicates a target item located closer to the lane entrance, whereas a larger value of r indicates a target item located deeper in the storage lane.

During this scan, if one or more non-target items are encountered before the next target item, these non-target items are temporarily recorded as blockers. When the next target item $i_{\ell,r}^{\text{target}}$ is encountered, all blockers that have not yet been assigned are associated with this target item. Let

$$\mathcal{I}_{\ell,r}^{\text{blocker}}$$

denote the set of blocking non-target items associated with target item $i_{\ell,r}^{\text{target}}$. The b -th blocking non-target item in storage lane ℓ is denoted by

$$i_{\ell,b}^{\text{blocker}}.$$

Thus, $\mathcal{I}_{\ell,r}^{\text{blocker}}$ contains the blocking non-target items that are located in front of $i_{\ell,r}^{\text{target}}$, must be relocated before this target item can be retrieved, and have not already been associated with another target item closer to the lane entrance.

For example, consider a local item arrangement in storage lane ℓ , from the lane entrance toward the deeper end, given by

$$i_{\ell,1}^{\text{blocker}}, i_{\ell,2}^{\text{blocker}}, i_{\ell,1}^{\text{target}}, i_{\ell,3}^{\text{blocker}}, i_{\ell,2}^{\text{target}}.$$

The corresponding blocker sets are

$$\mathcal{I}_{\ell,1}^{\text{blocker}} = \{i_{\ell,1}^{\text{blocker}}, i_{\ell,2}^{\text{blocker}}\}$$

and

$$\mathcal{I}_{\ell,2}^{\text{blocker}} = \{i_{\ell,3}^{\text{blocker}}\}.$$

This blocker-to-target association ensures that each blocking non-target item is counted only once when constructing the required-work lower bounds, thereby preventing the same relocation operation from being repeatedly included in the lower-bound calculations of multiple target items.

It should be noted that if a non-target item is located deeper than the last target item currently contained in the storage lane, and no target item exists behind it, this non-target item is not included in any $\mathcal{I}_{\ell,r}^{\text{blocker}}$. This is because relocating such an item is not necessarily required to retrieve any of the target items currently contained in the lane. Including such unnecessary relocation work could overestimate the minimum required work and would therefore violate the lower-bound property.

Target-specific required-work lower bound q .

For a target item $i_{\ell,r}^{\text{target}}$ in storage lane ℓ , its retrieval under the current storage configuration requires at least two types of unavoidable work. First, the retrieval operation of $i_{\ell,r}^{\text{target}}$ itself must be completed. Second, if blocking non-target items exist in front of the target item, all items in $\mathcal{I}_{\ell,r}^{\text{blocker}}$ must first be relocated.

Accordingly, the target-specific required-work lower bound associated with target item $i_{\ell,r}^{\text{target}}$ is defined as

$$q_{\ell,r} = \tau^{\text{retrieval}}(i_{\ell,r}^{\text{target}}) + \sum_{i_{\ell,b}^{\text{blocker}} \in \mathcal{I}_{\ell,r}^{\text{blocker}}} \tau^{\text{relocation}}(i_{\ell,b}^{\text{blocker}}). \quad (4.5)$$

In Eq. (4.5), $q_{\ell,r}$ denotes the minimum amount of required work associated with the r -th target item in storage lane ℓ ; $\tau^{\text{retrieval}}(i_{\ell,r}^{\text{target}})$ denotes the retrieval operation time lower bound of target item $i_{\ell,r}^{\text{target}}$; $\mathcal{I}_{\ell,r}^{\text{blocker}}$ denotes the set of blocking non-target items associated with this target item; and $\tau^{\text{relocation}}(i_{\ell,b}^{\text{blocker}})$ denotes the relocation operation time lower bound of blocking non-target item $i_{\ell,b}^{\text{blocker}}$.

If no blocking non-target item needs to be relocated before $i_{\ell,r}^{\text{target}}$ can be retrieved, i.e.,

$$\mathcal{I}_{\ell,r}^{\text{blocker}} = \emptyset,$$

the target-specific required-work lower bound reduces to

$$q_{\ell,r} = \tau^{\text{retrieval}}(i_{\ell,r}^{\text{target}}).$$

Hence, $q_{\ell,r}$ represents the minimum local amount of work that is unavoidably required to complete target item $i_{\ell,r}^{\text{target}}$, while temporarily disregarding the cumulative precedence effect of the other target items in the same storage lane. Compared with the retrieval operation time lower bound alone, $q_{\ell,r}$ explicitly incorporates the necessary relocation work induced by the blocking relationship in front of the corresponding target item.

Furthermore, both $\tau^{\text{retrieval}}$ and $\tau^{\text{relocation}}$ are evaluated according to the current system state. Therefore, if the shuttle is already located at the location of a particular item, or if a target item or blocking non-target item has already been loaded onto the shuttle, the movement or loading operations that have already been completed are not counted again in the corresponding operation time lower bound. Consequently, $q_{\ell,r}$ represents the minimum remaining amount of work required from the current state to complete the tasks associated with target item $i_{\ell,r}^{\text{target}}$, rather than the complete operation time recalculated from its initial storage state.

Precedence-constrained cumulative work lower bound R .

Although $q_{\ell,r}$ incorporates the necessary work associated with an individual target item and its direct blockers, it does not by itself fully represent the precedence relationship among different target items induced by the LIFO-type access constraint of a multi-deep storage lane.

Consider two target items in storage lane ℓ ,

$$i_{\ell,r-1}^{\text{target}} \quad \text{and} \quad i_{\ell,r}^{\text{target}},$$

ordered from the lane entrance toward the deeper end. Since $i_{\ell,r}^{\text{target}}$ is located deeper in the lane, its retrieval cannot be completed while bypassing the necessary preceding work associated with the items located closer to the lane entrance. More generally, to complete the r -th target item in storage lane ℓ , all unavoidable target-related work associated with the preceding part of the lane forms necessary preceding work.

Therefore, after the target-specific required-work lower bounds $q_{\ell,r}$ have been obtained, they are accumulated according to the target order from the lane entrance toward the deeper end. The precedence-constrained cumulative work lower bound is defined as

$$R_{\ell,r} = \sum_{h=1}^r q_{\ell,h}. \quad (4.6)$$

In Eq. (4.6), $R_{\ell,r}$ denotes the minimum cumulative amount of work required, from the current state, to complete the first r target items of storage lane ℓ while respecting the necessary in-lane access precedence, and $q_{\ell,h}$ denotes the target-specific required-work lower bound associated with the h -th target item in this storage lane.

Accordingly,

$$R_{\ell,1} = q_{\ell,1},$$

$$R_{\ell,2} = q_{\ell,1} + q_{\ell,2},$$

and, in general,

$$R_{\ell,r} = R_{\ell,r-1} + q_{\ell,r}. \quad (4.7)$$

In addition,

$$R_{\ell,0} = 0$$

is defined to represent the case in which no target item is completed from storage lane ℓ , such that this lane contributes no additional required work.

Consequently, for storage lane ℓ containing n_ℓ current target items, a sequence of precedence-constrained cumulative work lower bounds is obtained as

$$R_{\ell,1}, R_{\ell,2}, \dots, R_{\ell,n_\ell}.$$

Each $R_{\ell,r}$ represents the minimum cumulative work required to complete the first r target items in storage lane ℓ while preserving the necessary in-lane precedence relation.

From the perspective of lower-bound construction, q and R describe two different levels of information. The target-specific required-work lower bound $q_{\ell,r}$ captures the minimum local work associated with target item $i_{\ell,r}^{\text{target}}$ and its direct blocking non-target items, whereas the precedence-constrained cumulative work lower bound $R_{\ell,r}$ further incorporates the target-item precedence induced by the LIFO-type access constraint of the storage lane. Therefore, $R_{\ell,r}$ includes not only the required work associated with target item $i_{\ell,r}^{\text{target}}$ itself, but also all preceding target-related work that cannot be bypassed before this deeper target item can be completed.

For example, consider three target items in storage lane ℓ , ordered from the lane entrance toward the deeper end as

$$i_{\ell,1}^{\text{target}}, \quad i_{\ell,2}^{\text{target}}, \quad i_{\ell,3}^{\text{target}},$$

with the corresponding target-specific required-work lower bounds

$$q_{\ell,1}, \quad q_{\ell,2}, \quad q_{\ell,3}.$$

The corresponding precedence-constrained cumulative work lower bounds are

$$R_{\ell,1} = q_{\ell,1},$$

$$R_{\ell,2} = q_{\ell,1} + q_{\ell,2},$$

and

$$R_{\ell,3} = q_{\ell,1} + q_{\ell,2} + q_{\ell,3}.$$

Thus, if the third and deeper target item is to be completed, its minimum required work cannot be characterized solely by $q_{\ell,3}$. The unavoidable target-related work associated with the items located in front of it must also be considered. Therefore, $R_{\ell,3}$ provides a tighter cumulative work lower bound while preserving the precedence structure of the current storage lane.

Through the above construction, the operation-level lower bounds

$$\tau^{\text{retrieval}} \quad \text{and} \quad \tau^{\text{relocation}}$$

are first aggregated into the target-level target-specific required-work lower bound $q_{\ell,r}$, which is subsequently accumulated according to the deterministic in-lane precedence relation to obtain the precedence-constrained cumulative work lower bound $R_{\ell,r}$. The single-lane lower-bound construction can therefore be summarized as

$$\left\{ \tau^{\text{retrieval}}, \tau^{\text{relocation}} \right\} \longrightarrow q_{\ell,r} \longrightarrow R_{\ell,r}.$$

At this stage, each storage lane is represented by a set of precedence-constrained cumulative work lower bounds. In the subsequent derivation, the $R_{\ell,r}$ values across different storage lanes are combined while allowing the most favorable combination of completed target items among the lanes. This provides global completion-time lower bounds for different numbers of completed target items and ultimately enables the construction of a lower bound on the total tardiness of the current warehouse state.

4.3.3 Layout-level lower-bound derivation

In the previous subsection, the necessary access precedence induced by the Last-In, First-Out-type (LIFO-type) access constraint within an individual storage lane was exploited to first aggregate the lower bounds on individual retrieval and relocation operation times into the target-specific required-work lower bound $q_{\ell,r}$, and subsequently construct the precedence-constrained cumulative work lower bound $R_{\ell,r}$. Specifically,

$$q_{\ell,r} = \tau^{\text{retrieval}} \left(i_{\ell,r}^{\text{target}} \right) + \sum_{i_{\ell,b}^{\text{blocker}} \in \mathcal{I}_{\ell,r}^{\text{blocker}}} \tau^{\text{relocation}} \left(i_{\ell,b}^{\text{blocker}} \right)$$

represents the minimum required work directly associated with target item $i_{\ell,r}^{\text{target}}$, whereas

$$R_{\ell,r} = \sum_{h=1}^r q_{\ell,h}$$

further represents the minimum cumulative work that is unavoidable for completing the first r target items in storage lane ℓ while respecting the necessary in-lane access precedence.

However, $q_{\ell,r}$ and $R_{\ell,r}$ only characterize the local work structure within an individual storage lane, whereas the complete warehouse layout generally contains multiple storage lanes with target items to be processed. Unlike the necessary access precedence within the same storage lane, which is determined by the LIFO-type access constraint, the processing order among different storage lanes is not uniquely determined by the current physical layout and may vary depending on the scheduling policy. Therefore, to construct a layout-level lower bound that remains valid for any feasible scheduling policy, the specific

retrieval sequence of the target items is not predefined at the complete-layout level, nor is a particular completion rank associated with a specific target item. Instead, the derivation focuses on the minimum cumulative amount of work that is unavoidable for completing a given number of target items over all possible scheduling sequences.

To this end, let n^{target} denote the number of currently unfinished target items in the warehouse layout. For each completion rank

$$k = 1, 2, \dots, n^{\text{target}},$$

C_k is defined as the cumulative work lower bound that is unavoidable for completing any k of the current target items from the current system state under the corresponding constraint relaxation. Here, k represents only the number of completed target items and is not associated with any predefined specific target item. Therefore, C_k does not represent the operation-time lower bound of the k -th specific retrieval task. Instead, it represents the minimum cumulative amount of work that any feasible schedule must have performed before the k -th target-item completion can occur. In other words, C_k is a cumulative work lower bound associated with completion rank k , rather than the processing time of a particular target item.

Based on this common definition, three layout-level lower-bound calculation modes are proposed, namely `simple_shuttle_LB`, `mixed_shuttle_LB`, and `topology_shuttle_LB`. All three modes employ the same lower bounds on individual retrieval and relocation operation times and share the same construction of the target-specific required-work lower bound $q_{\ell,r}$ and the precedence-constrained cumulative work lower bound $R_{\ell,r}$. In addition, all three modes preserve the serial workload constraint imposed by the use of a single shuttle. That is, the work required by multiple retrieval-related tasks cannot be processed in parallel by multiple shuttles, but must instead be performed sequentially by the same shuttle. The fundamental difference among the three modes lies in the extent to which the precedence information induced by the LIFO-type access constraints of the individual storage lanes is preserved or relaxed when the single-lane lower bounds are aggregated into the layout-level cumulative work lower bounds C_k .

Specifically, `simple_shuttle_LB` applies the strongest relaxation to the precedence relations among target items within the same storage lane. It retains only the target-specific required work and the serial workload accumulation imposed by the single shuttle. Building on these constraints, `mixed_shuttle_LB` additionally exploits the lane-level precedence information contained in $R_{\ell,r}$ to tighten the corresponding cumulative work lower bounds, while still relaxing the complete combinatorial consistency between different precedence-constrained cumulative work lower bounds and the storage lanes to which they belong. Finally, `topology_shuttle_LB` preserves the complete precedence-constrained cumulative work structure induced by the LIFO-type access constraint within each storage lane and only allows the most favorable combination to be selected across different storage lanes for completing a given number of target items. Consequently, the three modes form a progressive layout-level lower-bound construction framework ranging from a stronger constraint relaxation to a more complete preservation of the storage-lane topology, yielding

$$C_k^{\text{simple}}, \quad C_k^{\text{mixed}}, \quad C_k^{\text{topology}}.$$

Simple shuttle lower-bound mode.

The `simple_shuttle_LB` mode retains two fundamental constraints. First, for each target item $i_{\ell,r}^{\text{target}}$, neither its own retrieval operation nor the necessary relocation operations of

its directly associated blocking non-target items can be omitted. Therefore, the corresponding target-specific required-work lower bound $q_{\ell,r}$ is fully retained. Second, because all retrieval-related operations in the considered system are executed by a single shuttle, the required work associated with different target items cannot be processed in parallel by multiple shuttles and must instead be performed serially by the same shuttle. Consequently, the minimum required work associated with multiple completed target items must be accumulated.

At the same time, **simple_shuttle_LB** completely relaxes the precedence relations among different target items within the same storage lane that are induced by the Last-In, First-Out-type (LIFO-type) access constraint. Under this relaxation, all target-specific required-work lower bounds $q_{\ell,r}$ in the entire layout are treated as independent required-work units. Neither the depth position of a target item relative to other target items in the same storage lane nor the target-related work located closer to the lane entrance is required to be considered before that target item is selected. Accordingly, the precedence-constrained cumulative structure contained in $R_{\ell,r}$ is not used to restrict the target-item completion order in this mode.

Based on this relaxation, all target-specific required-work lower bounds associated with the unfinished target items in the current warehouse layout are collected into the set

$$\mathcal{Q} = \left\{ q_{\ell,r} \mid i_{\ell,r}^{\text{target}} \text{ is an unfinished target item} \right\}.$$

Let n^{target} denote the number of unfinished target items in the current layout. Accordingly, $\mathcal{Q}$ contains n^{target} target-specific required-work lower bounds. These values are sorted in ascending order without considering their original storage-lane membership or their in-lane precedence positions, yielding

$$q_1^{\text{sorted}} \leq q_2^{\text{sorted}} \leq \dots \leq q_{n^{\text{target}}}^{\text{sorted}}.$$

Here, q_h^{sorted} denotes the h -th smallest target-specific required-work lower bound in the entire current layout. This sorting is performed solely according to the magnitude of the required-work lower bounds and does not preserve the original storage-lane membership or target-item precedence information. Therefore, under the simple-mode relaxation, a target item located deeper in a storage lane may be considered as an earlier completion candidate if its $q_{\ell,r}$ is sufficiently small, without first enforcing the precedence relations induced by other target items located closer to the lane entrance.

For each completion rank

$$k = 1, 2, \dots, n^{\text{target}},$$

the most favorable assumption is adopted to obtain the minimum cumulative work required for completing any k target items. Specifically, the shuttle is assumed to be able to process the k smallest target-specific required-work units in the entire layout first. The cumulative work lower bound under **simple_shuttle_LB** is therefore defined as

$$C_k^{\text{simple}} = \sum_{h=1}^k q_h^{\text{sorted}}. \quad (4.8)$$

Here, C_k^{simple} denotes the minimum cumulative amount of work that the single shuttle must perform before any k of the current target items can be completed under the simple relaxation; q_h^{sorted} denotes the h -th smallest target-specific required-work lower bound; and k denotes the corresponding completion rank.

Accordingly, for the first completion rank,

$$C_1^{\text{simple}} = q_1^{\text{sorted}},$$

and for the second completion rank,

$$C_2^{\text{simple}} = q_1^{\text{sorted}} + q_2^{\text{sorted}}.$$

More generally,

$$C_k^{\text{simple}} = C_{k-1}^{\text{simple}} + q_k^{\text{sorted}}.$$

This accumulation reflects the single-shuttle serial workload constraint. Even under the most favorable scheduling conditions, if k target items are to be completed, the required work associated with these target items cannot be executed simultaneously by multiple parallel resources. Instead, it must be performed sequentially by the same shuttle. Hence, the minimum cumulative work required before the k -th target-item completion cannot be smaller than the sum of the required-work lower bounds associated with the selected k target items.

Furthermore, because the simple mode does not prescribe a specific retrieval sequence, the most optimistic combination of k target items must be considered in order to preserve the lower-bound property for any feasible scheduling policy. Globally sorting all $q_{\ell,r}$ values and selecting the k smallest ones corresponds exactly to the minimum cumulative required work attainable after the target-to-target in-lane precedence relations have been completely relaxed. Therefore, for any feasible schedule that completes k target items, the cumulative work actually performed by the single shuttle cannot be smaller than C_k^{simple} .

From the perspective of constraint relaxation, `simple_shuttle_LB` can therefore be characterized as retaining the unavoidable required work associated with each target item and its direct blockers, as well as the serial workload accumulation imposed by the single shuttle, while completely relaxing the target-to-target precedence constraints induced by the storage-lane topology. Due to this relatively strong relaxation, C_k^{simple} generally provides the most optimistic completion-rank cumulative-work estimate among the three proposed modes and serves as the basis on which `mixed_shuttle_LB` and `topology_shuttle_LB` subsequently introduce additional storage-lane precedence information.

Mixed shuttle lower-bound mode.

The `mixed_shuttle_LB` mode builds upon the single-shuttle serial-workload lower bound constructed by `simple_shuttle_LB` and further introduces the precedence-constrained cumulative work lower bound $R_{\ell,r}$ within an individual storage lane to provide an additional lane-internal precedence constraint for each completion rank. Its purpose is not to replace the single-shuttle constraint retained by the simple mode with $R_{\ell,r}$, but rather to combine two independent necessary constraints, namely the single-shuttle serial workload constraint and the storage-lane precedence constraint, thereby obtaining a tighter cumulative work lower bound than that provided by the simple mode.

First, `simple_shuttle_LB` derives

$$C_k^{\text{simple}} = \sum_{h=1}^k q_h^{\text{sorted}},$$

where C_k^{simple} represents the minimum cumulative amount of work that the single shuttle must perform before any k of the current target items can be completed when the lane-level precedence relations among different target items are completely relaxed.

Therefore, C_k^{simple} mainly preserves two fundamental constraints: the unavoidable required work associated with each target item and its directly associated blockers, and the shared-resource constraint that all such required work must be performed serially by the same shuttle.

However, the simple mode treats all $q_{\ell,r}$ values in the entire layout as independent required-work units and allows them to be globally sorted without preserving the target-to-target precedence relations in their original storage lanes. Consequently, for a target item located deeper in a storage lane, the simple mode may regard it as an early completion candidate solely because its own $q_{\ell,r}$ is relatively small, without further requiring the target-related work located closer to the lane entrance to have already been completed. To complement this relaxation, the mixed mode additionally introduces the precedence-constrained cumulative work lower bound $R_{\ell,r}$.

For the r -th target item $i_{\ell,r}^{\text{target}}$ in storage lane ℓ ,

$$R_{\ell,r} = \sum_{h=1}^r q_{\ell,h}.$$

Here, $R_{\ell,r}$ represents the minimum cumulative amount of work that is unavoidable for completing the first r target items in storage lane ℓ while respecting the necessary access precedence induced by the Last-In, First-Out-type (LIFO-type) access constraint. Therefore, unlike $q_{\ell,r}$, which only represents the local required work associated with target item $i_{\ell,r}^{\text{target}}$ and its directly associated blockers, $R_{\ell,r}$ additionally incorporates all preceding target-related work that cannot be bypassed before the deeper target item can be completed.

It is important to note that $R_{\ell,r}$ is a lane-local lower bound. Its construction only characterizes the necessary cumulative work within storage lane ℓ and does not encode the cross-lane shared-resource coupling arising from multiple storage lanes competing for the same shuttle. Furthermore, $R_{\ell,r}$ inherits the relaxations already introduced in the lower bounds on individual retrieval and relocation operation times. For example, actual cross-lane repositioning, the specific relocation destination, cross-aisle congestion, route conflicts, and other factors that may increase the actual execution time are not fully considered. Therefore, $R_{\ell,r}$ itself is not a whole-layout single-shuttle cumulative-work bound, but rather a precedence-constrained local-work bound for an individual storage lane.

At the layout level, the mixed mode collects the $R_{\ell,r}$ values associated with all currently unfinished target items into the set

$$\mathcal{R} = \left\{ R_{\ell,r} \mid i_{\ell,r}^{\text{target}} \text{ is an unfinished target item} \right\}.$$

Let n^{target} denote the number of unfinished target items in the current layout. All $R_{\ell,r}$ values in $\mathcal{R}$ are then sorted in ascending order as

$$R_1^{\text{sorted}} \leq R_2^{\text{sorted}} \leq \dots \leq R_{n^{\text{target}}}^{\text{sorted}}.$$

Here, R_k^{sorted} denotes the k -th smallest precedence-constrained cumulative work lower bound in the entire layout. This global sorting no longer preserves the complete shared-resource coupling among the storage lanes to which the individual $R_{\ell,r}$ values belong. From a relaxation perspective, it can therefore be interpreted as a highly optimistic lane-parallel representation: the local precedence structure within each storage lane is preserved independently, whereas the resource competition among different storage lanes caused by sharing the same shuttle is relaxed in this R -based bound. This interpretation does not imply that the actual system or the lower-bound implementation assumes the physical

existence of multiple shuttles; rather, it describes the mathematical relaxation underlying the R -based bound.

Nevertheless, R_k^{sorted} remains a valid lower bound for the k -th completion rank. Consider any feasible schedule and let

$$\mathcal{I}_k^{\text{completed}}$$

denote the set of target items that have been completed when the k -th target-item completion occurs, with

$$|\mathcal{I}_k^{\text{completed}}| = k.$$

For any target item $i_{\ell,r}^{\text{target}} \in \mathcal{I}_k^{\text{completed}}$, the unavoidable preceding work represented by $R_{\ell,r}$ within its storage lane must already have been completed before that target item can be completed. Therefore, the actual cumulative work performed before the k -th completion must satisfy

$$C_k^{\text{actual}} \geq R_{\ell,r}, \quad \forall i_{\ell,r}^{\text{target}} \in \mathcal{I}_k^{\text{completed}}.$$

It follows that

$$C_k^{\text{actual}} \geq \max_{i_{\ell,r}^{\text{target}} \in \mathcal{I}_k^{\text{completed}}} R_{\ell,r}.$$

Moreover, for any subset of k values selected from all $R_{\ell,r}$ values in the current layout, the maximum value of this subset cannot be smaller than the globally k -th smallest value R_k^{sorted} . Hence,

$$\max_{i_{\ell,r}^{\text{target}} \in \mathcal{I}_k^{\text{completed}}} R_{\ell,r} \geq R_k^{\text{sorted}},$$

which yields

$$C_k^{\text{actual}} \geq R_k^{\text{sorted}}.$$

Therefore, R_k^{sorted} provides a completion-rank lower bound derived from the storage-lane precedence structure without prescribing a specific retrieval sequence.

However, this R -based lower bound does not replace the single-shuttle serial workload constraint retained by `simple_shuttle_LB`. Since the actual system contains only one shuttle, any feasible schedule must also satisfy

$$C_k^{\text{actual}} \geq C_k^{\text{simple}}.$$

Consequently, for every completion rank k , the actual cumulative work must simultaneously satisfy the two independent lower-bound constraints

$$C_k^{\text{actual}} \geq C_k^{\text{simple}}$$

and

$$C_k^{\text{actual}} \geq R_k^{\text{sorted}}.$$

Based on these two conditions, `mixed_shuttle_LB` combines the single-shuttle serial-workload lower bound provided by `simple_shuttle_LB` with the R -based lane-precedence lower bound and defines

$$C_k^{\text{mixed}} = \max \left\{ C_k^{\text{simple}}, R_k^{\text{sorted}} \right\}. \quad (4.9)$$

Here, C_k^{mixed} denotes the cumulative work lower bound that is unavoidable before the k -th target-item completion under the mixed mode; C_k^{simple} denotes the lower bound derived from the target-specific required work and the single-shuttle serial workload constraint; and R_k^{sorted} denotes the additional lower-bound constraint derived from the storage-lane precedence information.

Because both C_k^{simple} and R_k^{sorted} are valid lower bounds on C_k^{actual} , taking their maximum also preserves the lower-bound property. By definition,

$$C_k^{\text{mixed}} \geq C_k^{\text{simple}}.$$

If

$$R_k^{\text{sorted}} \leq C_k^{\text{simple}},$$

the lane-precedence constraint does not provide a stronger restriction than the single-shuttle serial-workload constraint at the corresponding completion rank, and therefore

$$C_k^{\text{mixed}} = C_k^{\text{simple}}.$$

Conversely, if

$$R_k^{\text{sorted}} > C_k^{\text{simple}},$$

the relaxation of the target-to-target in-lane precedence in the simple mode results in a more optimistic estimate of the cumulative work. In this case,

$$C_k^{\text{mixed}} = R_k^{\text{sorted}},$$

and the mixed mode tightens the simple bound using the additional lane precedence information.

Accordingly, `mixed_shuttle_LB` should not be interpreted as recovering the complete whole-layout topology or as replacing the original single-shuttle system with an actual multi-shuttle model. Instead, it retains the single-shuttle serial-workload lower bound provided by `simple_shuttle_LB` and supplements it with an independent lane-internal precedence constraint derived from R_k^{sorted} . This additional constraint can correct part of the overly optimistic estimation introduced by the complete relaxation of target-to-target in-lane precedence in the simple mode.

At the same time, the mixed mode still does not preserve the complete lane-wise combinatorial consistency across different storage lanes. When R_k^{sorted} is calculated, all $R_{\ell,r}$ values are treated as independent lane-local bounds, and the R values associated with different completion ranks are not required to jointly correspond to a globally consistent selection of target counts from the individual storage lanes. Therefore, the mixed mode preserves local lane-precedence information while still relaxing the complete combinatorial structure of these local precedence constraints at the whole-layout level. This remaining relaxation constitutes the principal distinction between the mixed mode and `topology_shuttle_LB`.

From the perspective of constraint relaxation, `mixed_shuttle_LB` can thus be interpreted as the combination of two complementary lower bounds:

$$\begin{aligned} C_k^{\text{simple}} &: \text{single-shuttle serial-workload constraint,} \\ R_k^{\text{sorted}} &: \text{lane-internal precedence constraint.} \end{aligned}$$

These two constraints are combined through

$$C_k^{\text{mixed}} = \max \left\{ C_k^{\text{simple}}, R_k^{\text{sorted}} \right\},$$

such that the tighter of the two lower-bound constraints is retained for each completion rank. The mixed mode can therefore be understood as a completion-rank-specific tightening of the simple mode using the lane-local precedence-constrained cumulative work lower bounds, rather than as a replacement of the original single-shuttle lower bound by an implicitly multi-shuttle model.

Topological shuttle lower-bound mode

The `topology_shuttle_LB` mode further preserves the complete precedence-constrained cumulative work structure within each storage lane on top of the relaxations adopted by `simple_shuttle_LB` and `mixed_shuttle_LB`. Unlike the mixed mode, which supplements each completion rank independently using the globally sorted R_k^{sorted} , the topology mode does not detach individual $R_{\ell,r}$ values from the storage lanes to which they belong. Instead, the association between each $R_{\ell,r}$, its corresponding storage lane, and the target-item depth order within that lane is retained throughout the layout-level lower-bound construction. Consequently, when constructing the cumulative work lower bound for completion rank k , the selected target items must jointly form a combination that is consistent with the precedence relations in all involved storage lanes.

For an arbitrary storage lane ℓ , let n_ℓ^{target} denote the number of currently unfinished target items in this lane. These target items are ordered from the lane entrance toward the deeper end as

$$i_{\ell,1}^{\text{target}}, i_{\ell,2}^{\text{target}}, \dots, i_{\ell,n_\ell^{\text{target}}}^{\text{target}}.$$

The corresponding precedence-constrained cumulative work lower bounds derived in the previous subsection are

$$R_{\ell,1}, R_{\ell,2}, \dots, R_{\ell,n_\ell^{\text{target}}},$$

with

$$R_{\ell,r} = \sum_{h=1}^r q_{\ell,h}.$$

In addition,

$$R_{\ell,0} = 0$$

is defined.

Therefore, selecting r completed target items from storage lane ℓ does not mean selecting any arbitrary r values of $q_{\ell,h}$. Instead, the selected target items must be the first r target items counted from the lane entrance, and the corresponding minimum required work is $R_{\ell,r}$. In other words, $R_{\ell,r}$ restricts the feasible target-selection structure within storage lane ℓ to a precedence-consistent prefix structure: if a deeper target item $i_{\ell,r}^{\text{target}}$ is included among the completed target items, all target-related required work associated with the target items located in front of it must also be included.

This constitutes the main distinction between the topology mode and the two preceding modes. In the simple mode, all $q_{\ell,r}$ values can be globally sorted across the entire layout, such that a deeper target item may be treated as an early completion candidate

independently of the target-related work located in front of it. In the mixed mode, although $R_{\ell,r}$ provides an additional lane-precedence constraint for each completion rank, the individual $R_{\ell,r}$ values are still globally sorted and used independently, and the R values associated with different completion ranks are not required to jointly correspond to a lane-wise consistent selection of target counts. The topology mode removes this remaining relaxation. For each completion rank k , the number of target items selected from every storage lane is explicitly considered, and the selected target items in each lane must respect the original precedence order.

Let n^{lane} denote the number of storage lanes that contain target-related work relevant to the current lower-bound calculation. Define

$$n_{\ell}^{\text{selected}}$$

as the number of target items selected for completion from storage lane ℓ when constructing a lower bound for a particular completion rank. Then,

$$n_{\ell}^{\text{selected}} \in \{0, 1, \dots, n_{\ell}^{\text{target}}\}.$$

Selecting $n_{\ell}^{\text{selected}}$ target items from storage lane ℓ means completing the first $n_{\ell}^{\text{selected}}$ target items closest to the lane entrance. Accordingly, the minimum required work contributed by storage lane ℓ is

$$R_{\ell, n_{\ell}^{\text{selected}}}.$$

For completion rank k at the layout level, the total number of selected target items over all storage lanes must be exactly k , i.e.,

$$\sum_{\ell=1}^{n^{\text{lane}}} n_{\ell}^{\text{selected}} = k.$$

For any lane-wise target-count combination satisfying this condition, the corresponding cumulative required-work lower bound is

$$\sum_{\ell=1}^{n^{\text{lane}}} R_{\ell, n_{\ell}^{\text{selected}}}.$$

This summation continues to reflect the single-shuttle serial workload constraint of the actual system. Although the processing sequence among different storage lanes is not predetermined, once the number of target items to be completed from each lane is specified, the corresponding minimum required work from all selected lanes must ultimately be performed by the same shuttle. Hence, the required work contributed by different storage lanes must be accumulated rather than treated as if it could be executed in parallel.

However, to ensure that the resulting quantity remains a lower bound independent of any particular scheduling policy, the topology mode does not prescribe which storage lane should be processed first, nor does it fix the actual retrieval sequence across different storage lanes. Instead, for a given completion rank k , the most favorable lane-wise target-count combination is selected among all combinations satisfying

$$\sum_{\ell=1}^{n^{\text{lane}}} n_{\ell}^{\text{selected}} = k.$$

Accordingly, the cumulative work lower bound under `topology_shuttle_LB` is defined as

$$C_k^{\text{topology}} = \min_{\substack{0 \leq n_\ell^{\text{selected}} \leq n_\ell^{\text{target}} \\ \sum_{\ell=1}^{n^{\text{lane}}} n_\ell^{\text{selected}} = k}} \sum_{\ell=1}^{n^{\text{lane}}} R_{\ell, n_\ell^{\text{selected}}}. \quad (4.10)$$

Here, C_k^{topology} denotes the minimum cumulative amount of work that the single shuttle must perform, from the current state, before any k target items can be completed while preserving the necessary precedence structure within every storage lane; n^{lane} denotes the number of storage lanes involved in the current lower-bound calculation; n_ℓ^{target} denotes the number of currently unfinished target items in storage lane ℓ ; n_ℓ^{selected} denotes the number of target items selected for completion from storage lane ℓ ; and $R_{\ell, n_\ell^{\text{selected}}}$ denotes the precedence-constrained cumulative work lower bound required to complete the first n_ℓ^{selected} target items in that lane.

For example, consider a layout containing three storage lanes and suppose that the cumulative work lower bound for the second completion rank is to be constructed, i.e., $k = 2$. Provided that the corresponding lanes contain sufficient target items, the feasible lane-wise target-count combinations include

$$(2, 0, 0), \quad (0, 2, 0), \quad (0, 0, 2), \quad (1, 1, 0), \quad (1, 0, 1), \quad (0, 1, 1).$$

If

$$(2, 0, 0)$$

is selected, both completed target items originate from the first storage lane. Because the precedence relation within this lane must be respected, the corresponding cumulative required work is

$$R_{1,2}.$$

If

$$(1, 1, 0)$$

is selected, one target item closest to the lane entrance is completed from each of the first and second storage lanes, and the corresponding cumulative required work is

$$R_{1,1} + R_{2,1}.$$

The remaining feasible combinations can be evaluated in the same manner. Consequently, the topology mode selects the minimum value among all feasible combinations:

$$C_2^{\text{topology}} = \min \{R_{1,2}, R_{2,2}, R_{3,2}, R_{1,1} + R_{2,1}, R_{1,1} + R_{3,1}, R_{2,1} + R_{3,1}\},$$

where only combinations permitted by the current number of target items in each storage lane are included.

This minimization has two simultaneous but conceptually distinct effects. On the one hand, for each individual storage lane, only one valid state from

$$R_{\ell,0}, R_{\ell,1}, \dots, R_{\ell, n_\ell^{\text{target}}}$$

may be selected. Therefore, target-related work located closer to the lane entrance cannot be bypassed, as in the simple mode, and individual globally sorted R_k^{sorted} values are not used independently, as in the mixed mode. Hence, every candidate combination

preserves the LIFO-type access precedence within all selected storage lanes. On the other hand, the allocation of the k completed target items across different storage lanes is still chosen optimistically through global minimization. This avoids prescribing any specific scheduling sequence across the storage lanes and thereby preserves the optimistic nature required for a valid layout-level lower bound.

From another perspective, the topology mode searches for the minimum cumulative required work among all precedence-consistent subsets of k target items. In the simple mode, any k target items in the entire layout may be selected. In the topology mode, an additional restriction is imposed: if the selected subset contains the r -th target item $i_{\ell,r}^{\text{target}}$ in storage lane ℓ , then the target items indexed $1, \dots, r-1$ in the same lane must also be represented through the corresponding preceding required work. Therefore, the set of feasible target-item combinations under the topology mode is a subset of that under the simple mode. Using the same $q_{\ell,r}$ values, it follows that

$$C_k^{\text{topology}} \geq C_k^{\text{simple}}.$$

This relation does not imply that the topology mode overestimates the actual cumulative work. Rather, the simple mode permits additional idealized combinations that violate the physical in-lane precedence, whereas the topology mode removes these infeasible combinations and thereby yields a tighter lower bound.

The topology mode also retains the lane-precedence information exploited by the mixed mode. For any feasible lane-wise target-count combination corresponding to completion rank k , the resulting cumulative required work cannot be smaller than the $R_{\ell,r}$ value associated with any completed target item contained in that combination. Therefore, the cumulative work must also satisfy the lower-bound constraint implied by the globally k -th smallest precedence-constrained cumulative work lower bound,

$$C_k^{\text{topology}} \geq R_k^{\text{sorted}}.$$

Together with

$$C_k^{\text{topology}} \geq C_k^{\text{simple}},$$

this yields

$$C_k^{\text{topology}} \geq \max \left\{ C_k^{\text{simple}}, R_k^{\text{sorted}} \right\}.$$

Since the mixed mode is defined as

$$C_k^{\text{mixed}} = \max \left\{ C_k^{\text{simple}}, R_k^{\text{sorted}} \right\},$$

the three cumulative work lower bounds satisfy

$$C_k^{\text{simple}} \leq C_k^{\text{mixed}} \leq C_k^{\text{topology}}.$$

These inequalities describe the progressive tightening of the cumulative work lower bound as more storage-lane precedence information is preserved. They should not be interpreted as strict inequalities, because two or all three modes may yield identical values for particular system states and completion ranks.

In practice, explicitly enumerating all combinations

$$\left(n_1^{\text{selected}}, n_2^{\text{selected}}, \dots, n_{n^{\text{lane}}}^{\text{selected}} \right)$$

becomes increasingly expensive as the number of storage lanes and target items grows. Therefore, the current implementation computes the topology lower bound using dynamic programming. The sequences

$$R_{\ell,0}, R_{\ell,1}, \dots, R_{\ell,n_{\ell}^{\text{target}}}$$

are merged one storage lane at a time. When a new storage lane is introduced, the algorithm considers the possible numbers of target items selected from that lane, evaluates their contribution to each completion rank, and retains the minimum cumulative required work for every rank. After all storage lanes have been processed, the procedure simultaneously obtains

$$C_1^{\text{topology}}, C_2^{\text{topology}}, \dots, C_{n^{\text{target}}}^{\text{topology}}.$$

This dynamic-programming procedure improves the computational efficiency of the lane-wise combination problem without changing the mathematical definition of the lower bound.

It should be emphasized that the term “topology” in `topology_shuttle_LB` does not imply that all physical routing and traffic constraints of the actual warehouse layout are preserved. The main additional topology information retained by this mode is the target-level precedence structure within each storage lane induced by the current item arrangement and the LIFO-type access constraint. The underlying $\tau^{\text{retrieval}}$, $\tau^{\text{relocation}}$, $q_{\ell,r}$, and $R_{\ell,r}$ still inherit the operation-level relaxations introduced previously. For example, actual cross-lane repositioning, specific relocation destinations, cross-aisle congestion, route conflicts, waiting times, and other factors that may increase the actual execution time are not fully represented. In addition, the actual switching sequence and travel continuity across different storage lanes are not fixed; instead, the most favorable lane-wise combination is retained at the layout-level lower-bound stage.

Therefore, `topology_shuttle_LB` should not be interpreted as a full simulation of the actual single-shuttle scheduling process. Rather, it preserves the storage-lane internal precedence structure more completely than the simple and mixed modes while retaining the relaxations required for constructing a valid lower bound. Its main characteristics can be summarized as

- retaining target-specific unavoidable work,
- retaining single-shuttle serial workload accumulation,
- fully preserving the precedence-consistent prefix structure within each storage lane,
- selecting the most favorable target-count combination across different storage lanes.

Accordingly, the progressive relationship among the three layout-level modes can be summarized as

- Simple: global selection of independent q ,
- Mixed: simple bound + an independent R -based precedence constraint,
- Topology: joint lane-wise combination of complete R structures.

Thus, `topology_shuttle_LB` removes the lane-wise combinatorial relaxation introduced by globally and independently sorting the $R_{\ell,r}$ values in the mixed mode. By jointly optimizing over the complete precedence-constrained cumulative work structures of all storage lanes, it constructs, for each completion rank k , the cumulative work lower bound C_k^{topology} that preserves the most complete storage-lane topology information among the three proposed modes.

4.3.4 State-level total-tardiness lower bound derivation

In the preceding layout-level lower-bound derivation, the cumulative work lower bounds corresponding to each completion rank k were constructed under the three proposed lower-bound modes, namely

$$C_k^{\text{simple}}, \quad C_k^{\text{mixed}}, \quad C_k^{\text{topology}}.$$

These quantities represent the minimum cumulative amount of work that is unavoidable, under the corresponding constraint relaxation, before any k of the current target items can be completed from the current system state. However, C_k^m only characterizes the minimum work required to complete different numbers of target items and does not yet explicitly incorporate the due dates of the target items. Therefore, it cannot directly represent a lower bound on the optimization objective considered in this study, namely total tardiness. To derive a state-level total-tardiness lower bound, the completion-rank cumulative work lower bounds are further combined with the due dates of the currently unfinished target items.

Let the current system state be denoted by s , and let

$$t(s)$$

denote the current system time. Assume that the current state contains

$$n^{\text{target}}(s)$$

unfinished target items. The due dates of these target items are sorted in ascending order as

$$d_{(1)}^{\text{due}}(s) \leq d_{(2)}^{\text{due}}(s) \leq \dots \leq d_{(n^{\text{target}}(s))}^{\text{due}}(s).$$

Here, $d_{(k)}^{\text{due}}(s)$ denotes the k -th smallest due date among all unfinished target items in state s . The parenthesized subscript (k) denotes the position of the corresponding due date in the sorted due-date sequence, while the superscript due identifies the quantity as a due date.

For any lower-bound mode

$$m \in \{\text{simple}, \text{mixed}, \text{topology}\},$$

$C_k^m(s)$ denotes the cumulative work lower bound required to complete any k of the currently unfinished target items from state s . Consequently, even under the most favorable subsequent scheduling policy, the k -th target-item completion cannot occur earlier than

$$t(s) + C_k^m(s).$$

Here, $t(s)$ represents the amount of system time that has already elapsed, whereas $C_k^m(s)$ represents the minimum remaining cumulative work required from the current state before the k -th target-item completion can occur. Their sum therefore constitutes a lower bound on the earliest possible completion time associated with completion rank k .

It should be noted that the layout-level lower-bound construction does not predetermine which specific target item will occupy completion rank k . Accordingly, $C_k^m(s)$ cannot be directly associated with the due date of a predefined target item. To preserve the optimistic nature required for a valid lower bound, the completion ranks and the due dates of

the currently unfinished target items must be paired in the most favorable manner with respect to total tardiness.

Since the completion-time lower bounds are ordered by completion rank, an optimistic assignment is obtained by pairing earlier completion ranks with smaller due dates. Therefore, the k -th completion-time lower bound

$$t(s) + C_k^m(s)$$

is paired with the k -th smallest due date

$$d_{(k)}^{\text{due}}(s).$$

The unavoidable tardiness lower bound associated with completion rank k is then given by

$$\max \left\{ 0, t(s) + C_k^m(s) - d_{(k)}^{\text{due}}(s) \right\}.$$

In this expression, $t(s) + C_k^m(s)$ denotes the earliest possible completion time of the k -th completed target item under lower-bound mode m , while $d_{(k)}^{\text{due}}(s)$ denotes the k -th smallest due date among the currently unfinished target items. If the earliest possible completion time is no later than the corresponding due date, no tardiness is unavoidable for that completion rank, and the contribution is therefore truncated to zero through the maximum operator.

In addition to the tardiness that may unavoidably be incurred by the unfinished target items, the current state may already contain target items that have been completed and unloaded at an Input/Output (I/O) point. For these completed tasks, their actual tardiness has already been realized and cannot be affected by any subsequent scheduling decision. Therefore, this part of the total tardiness does not require further lower-bound estimation and is directly included using its realized value.

Let

$$T^{\text{done}}(s)$$

denote the cumulative actual tardiness of all target items that have already been completed in state s . The state-level total-tardiness lower bound under lower-bound mode m is then defined as

$$LB^m(s) = T^{\text{done}}(s) + \sum_{k=1}^{n^{\text{target}}(s)} \max \left\{ 0, t(s) + C_k^m(s) - d_{(k)}^{\text{due}}(s) \right\}. \quad (4.11)$$

Here, $LB^m(s)$ denotes the total-tardiness lower bound of state s under lower-bound mode m ; $T^{\text{done}}(s)$ denotes the realized cumulative tardiness of the already completed target items; $n^{\text{target}}(s)$ denotes the number of currently unfinished target items; $t(s)$ denotes the current system time; $C_k^m(s)$ denotes the cumulative work lower bound required to complete any k of the current target items under mode m ; and $d_{(k)}^{\text{due}}(s)$ denotes the k -th smallest due date among the currently unfinished target items.

Accordingly, the state-level total-tardiness lower bound can be decomposed into two conceptually different components:

$$LB^m(s) = \underbrace{T^{\text{done}}(s)}_{\text{realized and irreversible tardiness}} + \underbrace{\sum_{k=1}^{n^{\text{target}}(s)} \max \left\{ 0, t(s) + C_k^m(s) - d_{(k)}^{\text{due}}(s) \right\}}_{\text{lower bound on future unavoidable tardiness}}.$$

The first component represents the scheduling cost that has already been realized and can no longer be changed, whereas the second component provides an optimistic estimate of the minimum additional tardiness that is unavoidable from the current state. This estimate jointly reflects the current warehouse configuration, the current system time, the unfinished target items and their due dates, as well as the workload and precedence information retained by the corresponding lower-bound mode.

Therefore, $LB^m(s)$ characterizes not only the tardiness that has already accumulated, but also the minimum future tardiness implied by the current system state before all remaining target items have actually been completed. This enables the lower bound to reflect changes in the future scheduling difficulty caused by changes in the current layout even before those changes are fully manifested as realized tardiness.

The three proposed lower-bound modes employ the same transformation from completion-rank cumulative work lower bounds to the state-level total-tardiness lower bound. Their only difference at this stage lies in the corresponding values of

$$C_k^{\text{simple}}, \quad C_k^{\text{mixed}}, \quad C_k^{\text{topology}}.$$

Accordingly, three state-level total-tardiness lower bounds are obtained:

$$LB^{\text{simple}}(s), \quad LB^{\text{mixed}}(s), \quad LB^{\text{topology}}(s).$$

As established in the preceding derivation,

$$C_k^{\text{simple}}(s) \leq C_k^{\text{mixed}}(s) \leq C_k^{\text{topology}}(s).$$

Since

$$\max\{0, t + C - d\}$$

is monotonically nondecreasing with respect to C , the corresponding state-level total-tardiness lower bounds satisfy

$$LB^{\text{simple}}(s) \leq LB^{\text{mixed}}(s) \leq LB^{\text{topology}}(s).$$

These inequalities indicate that the state-level total-tardiness lower bound can be progressively tightened as more storage-lane precedence information is preserved. They should not be interpreted as strict inequalities, since two or all three lower-bound modes may yield identical values for particular system states when the additional precedence constraints do not further increase the unavoidable tardiness estimate.

4.3.5 Reward construction based on the state-level lower bound

In the preceding subsection, the state-level total-tardiness lower bounds under the three proposed lower-bound modes were constructed as

$$LB^{\text{simple}}(s), \quad LB^{\text{mixed}}(s), \quad LB^{\text{topology}}(s).$$

These state-level lower bounds incorporate not only the realized tardiness that has already occurred and can no longer be changed, but also an estimate of the future unavoidable tardiness based on the due dates of the currently unfinished target items and the corresponding completion-rank cumulative work lower bounds $C_k^m(s)$. Therefore, compared with a reward constructed solely from realized tardiness, $LB^m(s)$ can reflect the potential influence of the current scheduling decision on future total tardiness at an earlier stage.

Specifically, in a multi-deep storage system, an upper-level decision may first change the accessibility of target items, the blocking relationships, the necessary relocation work, or the precedence structure within a storage lane, without immediately completing a target item. Consequently, if the reward is constructed only from changes in realized tardiness, a retrieval or relocation decision that substantially improves the future scheduling state may not receive timely feedback at the current decision point. In contrast, because $LB^m(s)$ changes with the current layout, the shuttle carrying state, the unfinished target items, and the remaining required work, the difference between the lower bounds of consecutive upper-level decision states can be used to convert the effect of the current action on future unavoidable total tardiness into more immediate feedback.

To provide a unified representation of the three lower-bound modes, let

$$m \in \{\text{simple, mixed, topology}\}.$$

Let s_t denote the system state at the t -th upper-level decision point, and let the upper-level agent select action a_t in this state. After the retrieval or relocation subtask associated with this upper-level action has been completed, the system reaches the next upper-level decision state s_{t+1} . Without considering the counterfactual correction associated with dynamic target activation at this stage, the reward for the upper-level action is constructed from the change in the state-level total-tardiness lower bound between these two states.

Since $LB^m(s)$ represents a cost-like quantity to be minimized, a smaller lower bound corresponds to a more favorable system state. To make the direction of state evaluation consistent with the reward-maximization objective in reinforcement learning, the potential function of state s under lower-bound mode m is defined as

$$\Phi^m(s) = -LB^m(s).$$

Here, $\Phi^m(s)$ denotes the potential value of state s under mode m , and $LB^m(s)$ denotes the corresponding state-level total-tardiness lower bound. The negative sign reverses the direction of the cost quantity such that a lower total-tardiness lower bound corresponds to a higher state potential. Accordingly, for any two states s^A and s^B , if

$$LB^m(s^A) < LB^m(s^B),$$

then

$$\Phi^m(s^A) > \Phi^m(s^B).$$

Thus, a state with a lower amount of unavoidable total tardiness is assigned a higher potential value.

In general, the potential difference between two consecutive states can be expressed as

$$\gamma\Phi^m(s_{t+1}) - \Phi^m(s_t),$$

where γ denotes the discount factor. In the current training setting,

$$\gamma = 1.$$

Therefore, the potential difference between two consecutive upper-level decision states reduces to

$$\Phi^m(s_{t+1}) - \Phi^m(s_t).$$

Substituting

$$\Phi^m(s) = -LB^m(s)$$

gives

$$\Phi^m(s_{t+1}) - \Phi^m(s_t) = LB^m(s_t) - LB^m(s_{t+1}).$$

Accordingly, the reward assigned to the t -th upper-level action under lower-bound mode m is defined as

$$r_t^m = LB^m(s_t) - LB^m(s_{t+1}).$$

Here, r_t^m denotes the lower-bound-based reward assigned to the t -th upper-level action; $LB^m(s_t)$ denotes the state-level total-tardiness lower bound before the action is executed; and $LB^m(s_{t+1})$ denotes the state-level total-tardiness lower bound at the next upper-level decision state after the corresponding subtask has been completed.

If the current upper-level action reduces the state-level lower bound, i.e.,

$$LB^m(s_{t+1}) < LB^m(s_t),$$

then

$$r_t^m > 0.$$

This indicates that the action reduces the total tardiness that remains unavoidable from the new state and therefore receives a positive reward. Such a reduction may result from, for example, decreasing the necessary future relocation work, improving the accessibility of target items, completing part of the unavoidable required work, or reducing the minimum cumulative workload associated with subsequent completion ranks.

Conversely, if

$$LB^m(s_{t+1}) > LB^m(s_t),$$

then

$$r_t^m < 0,$$

indicating that the action moves the system to a state with a larger amount of unavoidable total tardiness. If

$$LB^m(s_{t+1}) = LB^m(s_t),$$

then

$$r_t^m = 0,$$

meaning that although the action changes the system state, it does not alter the future unavoidable total tardiness captured by the corresponding lower-bound mode.

Therefore, the reward does not directly evaluate whether an individual retrieval or relocation operation immediately completes a target item. Instead, it evaluates whether the upper-level action improves the potential of the overall system state. Using

$$\Phi^m(s) = -LB^m(s),$$

the reward can equivalently be expressed as

$$r_t^m = \Phi^m(s_{t+1}) - \Phi^m(s_t).$$

Hence, under the current setting of $\gamma = 1$, the lower-bound-based reward has the mathematical form of a potential-difference reward between consecutive upper-level decision states. It should be noted that this potential difference is directly used as the upper-level training reward rather than being added to the original tardiness reward as an additional shaping term. Therefore, the proposed design is more precisely characterized as a potential-difference reward based on the state-level total-tardiness lower bound, rather than as an additional potential-based shaping term applied to the original reward.

The proposed reward construction also has an important cumulative property. Consider a normally completed episode with the sequence of upper-level decision states

$$s_0, s_1, \dots, s_T.$$

Without considering additional truncation penalties or counterfactual corrections, the cumulative lower-bound-based reward over the episode is

$$\sum_{t=0}^{T-1} r_t^m = \sum_{t=0}^{T-1} [LB^m(s_t) - LB^m(s_{t+1})].$$

Because the intermediate lower-bound terms cancel telescopically, this expression reduces to

$$\sum_{t=0}^{T-1} r_t^m = LB^m(s_0) - LB^m(s_T).$$

This shows that the intermediate variations in the lower bound determine how the reward is distributed among the upper-level decisions, while the total episode reward remains equal to the difference between the initial and terminal state lower bounds.

When an episode terminates normally, all target items have been completed, and therefore

$$n^{\text{target}}(s_T) = 0.$$

At this point, the future unavoidable tardiness component of the state-level lower bound disappears, such that

$$LB^m(s_T) = T^{\text{done}}(s_T).$$

Since all target items have been completed at the terminal state, $T^{\text{done}}(s_T)$ is equal to the final total tardiness of the episode, denoted by

$$T^{\text{total}}.$$

Hence,

$$LB^m(s_T) = T^{\text{total}},$$

and therefore

$$\sum_{t=0}^{T-1} r_t^m = LB^m(s_0) - T^{\text{total}}.$$

For a fixed problem instance, the initial state s_0 and the corresponding lower bound $LB^m(s_0)$ remain unchanged across different scheduling policies. Therefore, under normal episode termination and in the absence of additional reward terms, maximizing the cumulative lower-bound-based reward is equivalent to minimizing the final total tardiness. At the same time, the changes in the intermediate state lower bounds redistribute the originally delayed total-tardiness objective into immediate feedback at multiple upper-level decision points, thereby alleviating the delayed-reward problem in long-horizon scheduling.

All three lower-bound modes employ the same reward construction mechanism. Their difference lies only in the amount of precedence information retained by the state evaluation function $LB^m(s)$. Accordingly, the rewards are given by

$$r_t^{\text{simple}} = LB^{\text{simple}}(s_t) - LB^{\text{simple}}(s_{t+1}),$$

$$r_t^{\text{mixed}} = LB^{\text{mixed}}(s_t) - LB^{\text{mixed}}(s_{t+1}),$$

and

$$r_t^{\text{topology}} = LB^{\text{topology}}(s_t) - LB^{\text{topology}}(s_{t+1}).$$

The corresponding potential functions are

$$\Phi^{\text{simple}}(s) = -LB^{\text{simple}}(s),$$

$$\Phi^{\text{mixed}}(s) = -LB^{\text{mixed}}(s),$$

and

$$\Phi^{\text{topology}}(s) = -LB^{\text{topology}}(s).$$

Consequently, the complete hierarchical construction of the lower-bound-based reward can be summarized as

$$\{\tau^{\text{retrieval}}, \tau^{\text{relocation}}\} \longrightarrow q_{\ell,r} \longrightarrow R_{\ell,r} \longrightarrow C_k^m \longrightarrow LB^m(s) \longrightarrow \Phi^m(s) \longrightarrow r_t^m.$$

The lower bounds on individual retrieval and relocation operation times provide the fundamental required-work estimates; $q_{\ell,r}$ aggregates the work associated with a target item and its necessary blockers; $R_{\ell,r}$ further incorporates the internal precedence of the storage lane; C_k^m aggregates these local lower bounds into layout-level cumulative work lower bounds for different completion ranks; $LB^m(s)$ combines them with the current time and due dates to obtain the state-level total-tardiness lower bound; and finally, the improvement in the state potential is converted into the immediate reward assigned to each upper-level action.

4.3.6 Counterfactual reward shaping

To reduce the random fluctuations introduced into upper-level action rewards by unpredictable dynamic target activation, this work further adopts counterfactual state evaluation for the three lower-bound-based reward modes and uses it to shape the basic state-level lower-bound reward. In this work, *counterfactual* specifically means that, at the state reached after the current upper-level transition, the actual physical warehouse layout, current time, shuttle state, and already realized tardiness are preserved, while the

items newly activated during the current transition are temporarily treated as non-target items only for the lower-bound evaluation used to calculate the current reward.

This design addresses the specific reward-assignment problem introduced by dynamic order-driven target activation. From the perspective of the current upper-level decision point, target activations occurring during the execution of an upper-level action constitute unpredictable exogenous state changes caused by the dynamic order process. An upper-level retrieval or relocation action generally requires multiple discrete time steps to complete. Therefore, between the action-start state s_t and the subsequent upper-level decision state s_{t+1} , one or more non-target items may be dynamically activated and assigned due dates. If the lower bound of the actual state s_{t+1} is directly used to evaluate the current action, the resulting reward reflects not only the effect of the action on the warehouse layout, blocking relationships, and remaining required work, but also the change in the target-item set caused by newly arriving orders. Consequently, even identical or similar scheduling actions may receive substantially different rewards depending on the target activations that happen to occur during their execution.

Consistent with the state-level total-tardiness lower bound derived above, for any lower-bound mode

$$m \in \{\text{simple, mixed, topology}\},$$

the lower bound of a state s is defined as

$$LB^m(s) = T^{\text{done}}(s) + \sum_{k=1}^{n^{\text{target}}(s)} \max \left\{ 0, t(s) + C_k^m(s) - d_{(k)}^{\text{due}}(s) \right\}.$$

Here, $LB^m(s)$ denotes the state-level total-tardiness lower bound of state s under lower-bound mode m ; $T^{\text{done}}(s)$ denotes the realized cumulative tardiness of the already completed target items; $n^{\text{target}}(s)$ denotes the number of currently unfinished target items; $t(s)$ denotes the current system time; $C_k^m(s)$ denotes the cumulative work lower bound required to complete any k of the currently unfinished target items under mode m ; and $d_{(k)}^{\text{due}}(s)$ denotes the k th smallest due date among the currently unfinished target items.

Without counterfactual reward shaping, after the upper-level agent selects action a_t in state s_t and the corresponding retrieval or relocation subtask is completed, the system reaches the next upper-level decision state s_{t+1} . The basic lower-bound-based reward is

$$r_t^m = LB^m(s_t) - LB^m(s_{t+1}),$$

where r_t^m denotes the reward assigned to the t th upper-level action under lower-bound mode m . This reward measures the change in the state-level total-tardiness lower bound between the states before and after the execution of the current upper-level action.

Let

$$\Delta\mathcal{T}_t$$

denote the set of items that are newly activated as target items during the upper-level transition from s_t to s_{t+1} , that is, from the beginning of the current upper-level decision until the associated retrieval or relocation subtask is completed.

Under the counterfactual mode, the actual physical state reached after the transition is retained, but a counterfactual state evaluation is performed solely for the current reward calculation. Specifically, for each item in $\Delta\mathcal{T}_t$, its actual physical location and its influence on the warehouse topology and blocking relationships are preserved, while its

newly assigned target status is temporarily ignored during the lower-bound calculation. The corresponding counterfactual state-level lower bound is denoted by

$$LB_{\text{cf}}^m(s_{t+1} \mid \Delta\mathcal{T}_t).$$

Here, the subscript cf denotes the counterfactual state evaluation. This quantity does not correspond to a separate physical warehouse state generated through an alternative environment transition. Instead, it represents an alternative lower-bound evaluation of the actual state s_{t+1} in which the target activations that occurred during the current transition are temporarily excluded from the target-item set.

Importantly, the items whose newly assigned target status is temporarily ignored are not removed from the warehouse. Therefore, if an item in $\Delta\mathcal{T}_t$ remains in a storage lane and blocks another active target item at state s_{t+1} , it is still treated as a non-target blocker in the counterfactual state evaluation and contributes to the corresponding relocation required-work lower bound. Hence, the counterfactual state evaluation removes only the immediate influence of the newly assigned target-task attributes on the reward of the current transition, while preserving the physical position and blocking effect of these items.

Accordingly, the counterfactual reward is defined as

$$r_{t,\text{cf}}^m = LB^m(s_t) - LB_{\text{cf}}^m(s_{t+1} \mid \Delta\mathcal{T}_t).$$

Here, $r_{t,\text{cf}}^m$ denotes the reward assigned to the t th upper-level action under lower-bound mode m after applying counterfactual reward shaping. The starting point of the reward remains the lower bound of the actual pre-action state s_t , whereas the post-action term is replaced by the lower bound obtained through the counterfactual state evaluation of s_{t+1} .

Equivalently, the counterfactual reward can be written as

$$r_{t,\text{cf}}^m = r_t^m + F_{t,\text{cf}}^m,$$

where the counterfactual shaping term is

$$F_{t,\text{cf}}^m = LB^m(s_{t+1}) - LB_{\text{cf}}^m(s_{t+1} \mid \Delta\mathcal{T}_t).$$

Here, $F_{t,\text{cf}}^m$ denotes the reward correction associated with target activations occurring during the current upper-level transition. This term separates the difference between the actual state evaluation and the counterfactual state evaluation from the direct evaluation of the current action, so that the resulting reward more specifically reflects the scheduling effect of the upper-level action itself.

If no new target activation occurs during the current upper-level transition,

$$\Delta\mathcal{T}_t = \emptyset,$$

then the counterfactual state evaluation is identical to the actual state evaluation, i.e.,

$$LB_{\text{cf}}^m(s_{t+1} \mid \emptyset) = LB^m(s_{t+1}),$$

and therefore

$$F_{t,\text{cf}}^m = 0$$

and

$$r_{t,\text{cf}}^m = r_t^m.$$

Thus, the counterfactual reward shaping modifies the immediate reward only when dynamic target activation actually occurs during the execution of an upper-level action.

The rationale behind this design is to distinguish, within the reward evaluation of the current transition, between state changes that can be influenced by the current scheduling action and unpredictable exogenous changes caused by random order arrivals. The counterfactual state evaluation does not remove the elapsed time consumed by the current action, nor does it remove the actual effects of the action on the shuttle position, item positions, blocking relationships, or remaining required work. Therefore, an inefficient action that consumes excessive time, increases future relocation work, or creates a less favorable blocking structure is still evaluated accordingly through the lower bound. The only component temporarily excluded is the immediate effect of newly introduced target-task attributes arising from target activations during the current transition.

Moreover, the counterfactual state evaluation is used only for the reward calculation of the current transition. The actual next state remains s_{t+1} , in which the items in $\Delta\mathcal{T}_t$ retain their actual target status and assigned due dates. These newly activated target items therefore participate normally in the state representation, action space, and lower-bound calculation of subsequent upper-level decisions. In particular, the starting lower bound of the next transition is again evaluated using the actual state,

$$LB^m(s_{t+1}),$$

rather than

$$LB_{\text{cf}}^m(s_{t+1} \mid \Delta\mathcal{T}_t).$$

Therefore, the proposed reward shaping does not modify the actual evolution of the dynamic target-activation process and does not cause newly arriving orders to be ignored in subsequent decisions. Its role is limited to preventing the lower-bound change caused by unpredictable target activations during the execution of an upper-level action from being directly attributed to that action, thereby reducing the random fluctuations introduced by the dynamic order process into the action-level reward signal.

4.4 Der Vortrag

Die Folien können in L^AT_EX, Powerpoint, CorelDRAW oder von Hand erstellt werden. Eine Vorlage in Powerpoint ist unter `_Vorlagen_INM` abgelegt. Folgende Punkte sind generell zu beachten:

- Im Vorfeld des Vortrages ist mit dem Hauptbetreuer abzuklären wie viel Sprechzeit erlaubt ist und wann der Vortrag stattfindet. Die Sprechzeit darf auf keinen Fall überzogen werden.
- Machen Sie nicht zu viele Folien. Der Hauptbetreuer soll die Anzahl und die Qualität der Folien überprüfen.
- Die Titelfolie beinhaltet mindestens:
 - ◇ Titel der Studienarbeit
 - ◇ Name des Studierenden
 - ◇ Typ der Arbeit (z.B. Masterarbeit)
 - ◇ Datum
 - ◇ Universitäts- und Institutslogo

- ◊ Die Namen aller Betreuer
- Achten Sie darauf, dass Ihre Folien (inklusive Grafiken) durch den Beamer gut angezeigt werden. Bearbeiten Sie gegebenenfalls Ihre Folien.

Bibliography

- [1] ResearchAndMarkets.com. B2c ecommerce global market size & forecast by value and volume across 80+ kpis – databook q4 2025 update. Technical report, ResearchAndMarkets.com, November 2025. Report ID: 6191206.
- [2] Zuopeng Xiao, Quan Yuan, Yonghai Sun, and Xixiong Sun. New paradigm of logistics space reorganization: E-commerce, land use, and supply chain management. *Transportation Research Interdisciplinary Perspectives*, 9:100300, 2021.
- [3] Takanori Sakai, Kohei Santo, Shinya Tanaka, and Tetsuro Hyodo. Locations of logistics facilities for e-commerce: A case of the tokyo metropolitan area. *Research in Transportation Business & Management*, 56:101174, 2024.
- [4] Jakob Marolt, Nenad Kosanić, and Tone Lerher. Relocation and storage assignment strategy evaluation in a multiple-deep tier-captive automated vehicle storage and retrieval system with undetermined retrieval sequence. *The International Journal of Advanced Manufacturing Technology*, 118:3403–3420, 2022.
- [5] Amazon. Amazon sets new prime delivery speed record in 2025 with over 13 billion items arriving the same or next day around the world, February 2026. Press release, February 3, 2026.
- [6] Ali Ahmed, Amit Deokar, Ho Cheung Brian Lee, and Nichalin Summerfield. The role of commitment in online reputation systems: An empirical study of express delivery promise in an e-commerce platform. *Decision Support Systems*, 176:114061, 2024.
- [7] Haley Paluzzi, Haozhe Chen, Michael Howe, Patricia J. Daugherty, and Travis Tokar. Consumer impatience in the e-commerce home delivery context: a consumer-centric supply chain perspective for time-based competition. *International Journal of Physical Distribution & Logistics Management*, 55(1):1–21, 2025.
- [8] Wenquan Dong, Mingzhou Jin, Yanyan Wang, and Peter Kelle. Retrieval scheduling in crane-based 3d automated retrieval and storage systems with shuttles. *Annals of Operations Research*, 302(1):111–135, 2021.
- [9] Jingjing Yang, René B. M. de Koster, Xiaolong Guo, and Yugang Yu. Scheduling shuttles in deep-lane shuttle-based storage systems. *European Journal of Operational Research*, 308(2):696–708, 2023.
- [10] Chenhao Ren, Qing Yan, and Zhi Liu. Scheduling optimisation in a multi-deep tier-to-tier four-way shuttle storage and retrieval system. *Computers & Industrial Engineering*, 204:111095, 2025.
- [11] Zhi Liu, Jiansha Lu, Jun Chen, Zhilong Xu, and Yingde Li. Scheduling optimization of hybrid multi-deep robotic mobile fulfillment systems. *International Journal of Production Research*, 64(6):2372–2395, 2026.

- [12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [14] Jialin Lu, Jing Yang, Shaobo Li, Yijun Li, Wu Jiang, Jiangtian Dai, and Jianjun Hu. A2C-DRL: Dynamic scheduling for stochastic edge-cloud environments using A2C and deep reinforcement learning. *IEEE Internet of Things Journal*, 11(9):16915–16926, 2024.
- [15] Xiao Mao, Guohua Wu, Mingfeng Fan, Zhiguang Cao, and Witold Pedrycz. DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV. *IEEE Transactions on Automation Science and Engineering*, 22:1028–1044, 2025.
- [16] Xinquan Wu, Xuefeng Yan, Donghai Guan, and Mingqiang Wei. A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time. *Engineering Applications of Artificial Intelligence*, 131:107790, 2024.
- [17] Jing He, Xinglu Liu, Qiyao Duan, Wai Kin Victor Chan, and Mingyao Qi. Reinforcement learning for multi-item retrieval in the puzzle-based storage system. *European Journal of Operational Research*, 305(2):820–837, 2023.
- [18] Zhun Xu, Liyun Xu, Huan Shao, Gang Shang, and Zhuo Zhou. Real-time transaction scheduling in multiple-deep tier-to-tier shuttle-based storage and retrieval systems by using deep q-learning. *Engineering Applications of Artificial Intelligence*, 165:113448, 2026.
- [19] Funing Li, Yuan Tian, Ruben Noortwyck, Jifeng Zhou, Liming Kuang, and Robert Schulz. Topology-aware and highly generalizable deep reinforcement learning for efficient retrieval in multi-deep storage systems. *Journal of Intelligent Manufacturing*, 2025.
- [20] Marco Caserta, Silvia Schwarze, and Stefan Voß. A mathematical formulation and complexity considerations for the blocks relocation problem. *European Journal of Operational Research*, 219(1):96–104, 2012.

Eigenständigkeitserklärung

Ich erkläre mit meiner Unterschrift, die Prüfungsordnung meines Studiengangs zur Kenntnis genommen, die vorliegende Arbeit selbständig verfasst und die im betroffenen Fachgebiet üblichen Zitiervorschriften eingehalten zu haben. Weiter versichere ich, dass das elektronische Exemplar mit den eingereichten gedruckten Exemplaren übereinstimmt.

Prüfungsordnungen:

www.uni-stuttgart.de/studieren/studium/admin/po/

Ort, Datum

Unterschrift